Studio Nx

P1-14 내 프로젝트 화면 JS — 함수 설명 합본 (JS 29~42)

2026-10-12

목차

1. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/29_myproject_formatRelativeTime.md
2. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/30_myproject_loadProjects.md
3. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/31_myproject_renderProjects.md
4. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/32_myproject_renderPager.md
5. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/33_myproject_initSearch.md
6. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/34_myproject_initRowMenus.md
7. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/35_studio_canOpenStudio.md
8. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/36_studio_showDesktopOnly.md
9. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/37_studio_openStudio.md
10. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/38_studio_initDesktopOnlyLinks.md
11. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/39_myproject_initNewProjectForm.md
12. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/40_myproject_initEditProjectForm.md
13. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/41_myproject_initDeleteProject.md
14. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/42_myproject_loadInitialList.md

JS 29 — `formatRelativeTime()`

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면: 목록의 "마지막 수정" 칸 글자 (PRD 01 FR-123)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	28 `initContactForm()` · 30 `loadProjects()`

1. 하는 일

서버가 준 수정 시각(`updated_at`)을 사람이 읽기 쉬운 글자로 바꿉니다. 목록 오른쪽의 "언제" 칸에 들어가는 글자입니다 (HTML 예시와 같은 모양).

수정한 지	글자
1분 안 (또는 미래: 컴퓨터 시계가 조금 느릴 때)	`Just now`
1분, 12분	`1 minute ago`, `12 minutes ago`
1시간, 11시간 (24시간 안)	`1 hour ago`, `11 hours ago`
달력으로 어제	`Yesterday`
2~7일 전	`2 days ago` … `7 days ago`
8일 이상이고 올해	`Sep 30`
다른 해	`Dec 31, 2025`

2. 언제 쓰나

호출하는 곳	화면
`renderProjects` (31, 다음 함수들)	목록 한 줄의 `[data-field="when"]` 칸입니다. 예: `Site Song … 12 minutes ago`

서버 시각의 모양: 서버는 UTC 시각을 **시간대 표시 없이** 보냅니다 (`"2026-10-08T02:10:11.123000"`). 자바스크립트의 `new Date()`는 시간대 표시가 없는 날짜·시각 문자열을 **이 컴퓨터의 시간대**로 읽으므로, 그대로 쓰면 서울에서는 9시간이 어긋납니다. 그래서 끝에 `Z`(UTC라는 표시)를 붙여서 읽습니다. userInfo의 가입일(`formatDate`, P1-1022)과 같은 방법입니다.

3. 코드

import (`myproject.js` 맨 위)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout } from '../core/auth.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio } from '../core/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

이 함수는 import를 쓰지 않습니다. 날짜 계산은 브라우저의 기본 기능(`Date`, `toLocaleDateString`)만으로 합니다. (import 정리는 다음 함수들에서 합니다.)

함수

```
export function formatRelativeTime(isoTime, now = new Date()) {
  const hasZone = /[zZ]$|[+-]\d\d:\d\d$/.test(isoTime);
  const time = new Date(hasZone ? isoTime : isoTime + 'Z');
  const minutes = Math.floor((now - time) / 60000);

  if (minutes < 1) return 'Just now';
  if (minutes < 60) return minutes === 1 ? '1 minute ago' : minutes + ' minutes ago';
  const hours = Math.floor(minutes / 60);
  if (hours < 24) return hours === 1 ? '1 hour ago' : hours + ' hours ago';

  // 달력으로 며칠 전인지 — 두 날의 자정끼리 비교 (시각은 버림)
  const today = new Date(now.getFullYear(), now.getMonth(), now.getDate());
  const day = new Date(time.getFullYear(), time.getMonth(), time.getDate());
  const days = Math.round((today - day) / 86400000);   // round: 서머타임으로 하루가 23·25시간인 날 대비
  if (days <= 1) return 'Yesterday';
  if (days <= 7) return days + ' days ago';

  const options = { month: 'short', day: 'numeric' };
  if (time.getFullYear() !== now.getFullYear()) options.year = 'numeric';
  return time.toLocaleDateString('en-US', options);
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
if (minutes < 60) return `${minutes} minute${minutes === 1 ? '' : 's'} ago`;
…
return time.toLocaleDateString('en-US', {
  month: 'short', day: 'numeric',
  ...(time.getFullYear() !== now.getFullYear() && { year: 'numeric' }),
});
```

최신 문법	실제 코드	뜻
`` `${minutes} minute…` `` (템플릿 문자열)	`minutes + ' minutes ago'`	문자열 안에 값을 넣습니다
`...(조건 && { year: 'numeric' })`	`if (조건) options.year = 'numeric'`	조건이 맞을 때만 칸을 추가합니다

문법 풀이

import 문법	뜻
(이 함수는 import 없음)	`Date`, `Math`, `toLocaleDateString`은 브라우저에 원래 있습니다

식	뜻	예
`now = new Date()` (매개변수 기본값)	넘기지 않으면 지금 시각입니다. 시험할 때만 시각을 넘깁니다	`f(iso)`는 지금 기준, `f(iso, 정오)`는 정오 기준
`/[zZ]$\|[+-]\d\d:\d\d$/.test(isoTime)`	끝이 `Z` 또는 `+09:00`처럼 시간대로 끝나는지 확인합니다	`"...02:48:00"` → false, `"...02:48:00Z"` → true
`now - time`	두 `Date`를 빼면 **밀리초** 차이가 나옵니다	12분 = 720000

식	뜻	예
`/ 60000`, `Math.floor`	밀리초를 분으로 바꾸고, 소수점을 버립니다	11분 59.9초 → 11
`new Date(년, 월, 일)`	그날 자정입니다 (이 컴퓨터의 시간대 기준)	시각을 버리고 날짜만 비교하려고 씁니다
`/ 86400000`, `Math.round`	밀리초를 일로 바꾸고(하루 = 86 400 000ms), 반올림합니다	서머타임이 바뀌는 날의 3.04 → 3
`toLocaleDateString('en-US', { month: 'short', day: 'numeric' })`	영어 날짜 글자를 만듭니다	`Sep 30`이고, year를 더하면 `Dec 31, 2025`

4. 흐름

1. 시간대 표시가 없으면 `Z`를 붙여 UTC로 읽습니다.
2. 지금과의 차이(분)를 구해서 1분 안, 60분 안, 24시간 안이면 그에 맞는 글자를 돌려줍니다.
3. 24시간이 넘으면 **달력** 기준으로 며칠 전인지 세서 어제, 7일 안, 그 밖에는 날짜를 돌려줍니다.

5. 왜 이렇게

선택	이유
`Z`를 붙여서 읽음	서버 시각은 UTC인데 표시가 없습니다. 붙이지 않으면 서울에서는 "9 hours ago"가 "Just now"로 나오는 식으로 어긋납니다
이미 시간대가 있으면 그대로 씀	나중에 서버가 `Z`를 붙여 보내더라도 `ZZ`가 되어 깨지지 않게 합니다
24시간 안은 **시간**으로, 넘으면 **달력**으로 셈	밤 12시 30분에 본 어젯밤 11시의 수정은 "Yesterday"보다 "1 hour ago"가 정확합니다. 하루가 넘으면 "어제, 3일 전"이 사람이 생각하는 방식입니다
미래 시각은 `Just now`	컴퓨터 시계가 서버보다 조금 느리면 방금 만든 프로젝트가 "-1 minutes ago"로 나올 수 있습니다
일 수는 `Math.round`	서머타임이 바뀌는 날은 하루가 23시간이나 25시간이므로 나누면 2.96이나 3.04가 나옵니다. 미국과 유럽 사용자에 대비한 것입니다
7일이 넘으면 날짜를, 다른 해면 연도까지 보여 줌	템플릿과 HTML 예시와 같습니다. "작년 12월 31일"을 "Dec 31"로만 쓰면 올해로 착각할 수 있습니다
`now` 매개변수	시험에서 "지금"을 고정하려고 둡니다. 화면에서는 넘기지 않습니다
`'en-US'`로 고정	시켠서 밖의 페이지는 영어만 씁니다 (2026-10-07 결정)

6. 시험 (실제 myProject.html, 헤드리스 Chrome, 20개)

페이지 안에서 `import('/static/js/pages/myproject.js')`로 함수를 불렀습니다. 시간대는 Chrome 설정으로 고정했습니다.

서울 (지금 = 2026-10-08 12:00 서울 = 03:00 UTC)

서버 시각 (UTC)	결과	확인한 것
`…03:00:00`, `…02:59:30`, `…03:05:00`	`Just now` ×3	같은 시각, 30초 전, 미래 5분
`…02:59:00`	`1 minute ago`	단수형
`…02:48:00`, `…02:47:59.500000`, `…02:48:00Z`, `…11:48:00+09:00`	`12 minutes ago` ×4	서버 형식(소수 6자리), `Z`, `+09:00`이 모두 같게 나옵니다

서버 시각 (UTC)	결과	확인한 것
`…02:48:00.123000`	`11 minutes ago`	11분 59.877초이므로 버림합니다
`…02:00:00`, `10-07T15:30:00`	`1 hour ago`, `11 hours ago`	서울로는 오늘 00:30입니다
`10-07T02:00:00`, `10-06T15:00:00`	`Yesterday` ×2	25시간 전, 서울 기준 어제 자정
`10-06T14:59:00`	`2 days ago`	서울 기준 그저께 23:59입니다 (1분 차이로 이틀 전)
`10-01T03:00:00`, `09-30T03:00:00`	`7 days ago`, `Sep 30`	7일까지는 "days ago", 8일부터는 날짜입니다
`2025-12-31T03:00:00`	`Dec 31, 2025`	다른 해

뉴욕 (서머타임이 끝나는 11월 1일 사이)

경우	결과
10월 31일 정오(EDT) → 11월 3일 정오(EST): 사이에 25시간짜리 하루가 있음	`3 days ago`
23.5시간 전 (날짜로는 어제)	`23 hours ago`
`now` 생략 (방금 시각)	`Just now`

처음 돌렸을 때 1개가 기대와 달랐는데, 함수가 아니라 **시험의 기대값**이 틀린 것이었습니다 (`02:48:00.123`은 11분 59.877초 전이므로 `11 minutes ago`가 맞습니다). 기대값을 고치고, 12분이 되는 서버 형식의 시각 (`02:47:59.500000`)을 따로 넣었습니다.

JS 30 — `loadProjects()`

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면: 목록 받기 (PRD 01 FR-123)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	29 `formatRelativeTime()` · 31 `renderProjects()`

1. 하는 일

검색어, 정렬, 페이지 조건으로 **내 프로젝트 목록을 서버에서 받고**, 그 조건을 기억해 **주소창에도 남깁니다**. 화면에 그리는 일은 31번 `renderProjects`가 합니다.

부른 값	서버 요청	주소창
`loadProjects()`	`/projects?query=&sort=updated&page=1&size=10`	`myProject.html`
`{ query: ' gam ', sort: 'title' }`	`query=gam&sort=title…`	`myProject.html?query=gam&sort=title`
`{ page: 2 }`	`page=2`	`myProject.html?page=2`
`{ sort: 'random', page: 'abc' }` (손으로 친 주소)	`sort=updated&page=1`로 고쳐서 보냅니다	`myProject.html`
`{ query: 'a&b%' }`	`query=a%26b%25` (안전한 글자로 바꿔 보냅니다)	`?query=a%26b%25`

실패	옮겨 가는 곳
401 (로그인이 풀림)	`home.html#login?next=/myProject.html`이며, 로그인한 뒤 돌아옵니다
500, 502 등 500번대	`errors/500.html` (503과 504는 같은 번호의 화면)
서버에 닿지 못함 (네트워크)	`errors/503.html`
그 밖의 4xx (예: 422)	`errors/400.html`

2. 언제 쓰나

호출하는 곳	화면
페이지 시작 (38)	처음 열 때 주소의 `?query=&sort=&page=`로 부릅니다
검색과 정렬 (33)	검색 칸에 입력하거나 정렬을 바꿀 때 부릅니다 (1페이지부터)
페이지 넘김 (32)	Prev, 1 2 3, Next
새 프로젝트를 만들거나 삭제한 뒤 (35, 37)	목록을 다시 받습니다. `current`의 조건을 그대로 씁니다

`history.replaceState`는 페이지를 새로 열지 않고 **주소창 글자만** 바꾸는 브라우저 기능입니다. 새로고침해도 같은 검색 결과가 나오고, 주소를 복사해서 보내도 같은 화면이 열립니다. `pushState`와 달리 뒤로 가기 기록을 쌓지 않으므로, 글자 하나를 칠 때마다 뒤로 가기 기록이 늘어나지 않습니다.

3. 코드

import (`myproject.js` 맨 위)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio } from '../core/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

`loginUrl`과 `api` 줄을 이번에 추가했습니다. 이 함수는 `api`(서버 요청, P1-02)와 `loginUrl`(로그인 주소 만들기, P1-02)을 씁니다.

상수 (이번에 추가)

```
/** 정렬 값 — #project-sort 의 option value · 서버 ProjectSort 와 같음 */
const SORTS = ['updated', 'title', 'created'];

/** 검색어 최대 길이 — 서버 query max_length 와 같음 */
const QUERY_MAX = 100;

/** 같은 번호의 오류 화면이 있는 상태 코드 (frontend/errors/) — userinfo.js 와 같음 */
const ERROR_PAGES = [400, 403, 404, 429, 500, 503, 504];

/** 지금 보고 있는 목록 조건 — 페이지 넘김 · 삭제 뒤 다시 받을 때 씀 (loadProjects 가 갱신) */
export const current = { query: '', sort: 'updated', page: 1 };
```

함수

```
export async function loadProjects({ query = '', sort = 'updated', page = 1 } = {}) {
  // 1. 값 정리 — 주소창에 손으로 친 값(?page=abc, ?sort=random)도 서버가 422 를 내지 않게
  query = String(query).trim().slice(0, QUERY_MAX);
  if (SORTS.indexOf(sort) < 0) sort = 'updated';
  page = parseInt(page, 10);
  if (!(page >= 1)) page = 1;   // NaN · 0 · 음수 → 1

  // 2. 서버에 요청
  const params = new URLSearchParams({ query: query, sort: sort, page: String(page), size: String(PAGE_S
IZE) });
  const ticket = ++loadTicket;
  let data;
  try {
    data = await api.get('/projects?' + params.toString());
  } catch (err) {
    if (ticket !== loadTicket) return null;   // 더 새 요청이 있으면 이 실패는 무시
    if (err.status === 401) {
      location.replace(loginUrl(location.pathname + location.search + location.hash));
    } else {
      let errorPage = err.status >= 500 ? 500 : 400;
      if (ERROR_PAGES.indexOf(err.status) >= 0) errorPage = err.status;
      else if (err.status === 0) errorPage = 503;
      location.replace('errors/' + errorPage + '.html');
```

```
    }
    return null;
  }

  if (ticket !== loadTicket) return null;   // 그사이 더 새 요청을 보냄 → 이 응답은 버림 (33)

  // 3. 지금 조건 기억 + 주소에 남김 (기본값은 빼서 주소를 짧게) — 새로고침 · 뒤로 가기에서 유지
  current.query = query;
  current.sort = sort;
  current.page = page;
  const shown = new URLSearchParams();
  if (query) shown.set('query', query);
  if (sort !== 'updated') shown.set('sort', sort);
  if (page !== 1) shown.set('page', String(page));
  const search = shown.toString();
  history.replaceState(null, '', location.pathname + (search ? '?' + search : '') + location.hash);

  return data;
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
const params = new URLSearchParams({ query, sort, page, size: PAGE_SIZE });
…
const shown = new URLSearchParams({
  ...(query && { query }),
  ...(sort !== 'updated' && { sort }),
  ...(page !== 1 && { page }),
});
history.replaceState(null, '', `${location.pathname}${search ? `?${search}` : ''}${location.hash}`);
if (!SORTS.includes(sort)) sort = 'updated';
```

최신 문법	실제 코드	뜻
`{ query, sort }` (줄임 속성)	`{ query: query, sort: sort }`	이름과 값이 같은 칸입니다
`...(조건 && { query })`	`if (조건) shown.set('query', query)`	조건이 맞을 때만 칸을 넣습니다
`SORTS.includes(sort)`	`SORTS.indexOf(sort) < 0`	목록에 있는지 확인합니다
템플릿 문자열 `` `${a}?${b}` ``	`a + '?' + b`	문자열을 잇습니다

문법 풀이

import 문법	뜻
`import { api } from '../core/api.js'`	`api.get(주소)`는 토큰을 붙여 보내고, 토큰이 만료되면 한 번 재발급한 뒤 다시 보냅니다. 실패하면 `ApiError`(status, code, message)를 던집니다 (P1-02)
`import { loginUrl } from '../core/auth.js'`	`loginUrl('/myProject.html')`은 `'home.html#login?next=%2FmyProject.html'`을 만듭니다 (P1-02, P1-07)

식	뜻	예
`{ query = '', sort = 'updated', page = 1 } = {}`	객체 하나를 받아서 칸을 꺼내고, 없으면 기본값을 씁니다. 아무것도 넘기지 않아도 됩니다	`loadProjects({ page: 2 })` → query는 `''`, sort는 `'updated'`

식	뜻	예
`String(query).trim().slice(0, QUERY_MAX)`	문자열로 바꾸고 앞뒤 공백을 지운 뒤 100자까지만 남깁니다	`' gam '` → `'gam'`
`parseInt(page, 10)`	문자열을 정수(10진수)로 바꿉니다	`'2'` → 2, `'abc'` → `NaN`
`!(page >= 1)`	1 이상이 **아니면** 참입니다. `NaN >= 1`은 false이므로 NaN도 걸러집니다	`NaN`, 0, −3 → 1로 바꿉니다
`new URLSearchParams({…}).toString()`	칸들을 `a=1&b=2` 모양으로 만들고, `&`, `%`, 공백은 안전한 글자로 바꿉니다	`'a&b%'` → `a%26b%25`
`err.status >= 500 ? 500 : 400`	조건 ? 맞을 때 : 아닐 때	502 → 500, 422 → 400
`location.replace(주소)`	그 주소로 옮겨 갑니다. 지금 페이지를 기록에서 **바꿔치기**하므로, 뒤로 가기로 오류 전의 페이지에 다시 오지 않습니다	
`export const current = {…}`	다른 함수(32, 35, 37)가 읽는 "지금 조건"입니다. `const`여도 안의 칸은 바꿀 수 있습니다	`current.page = 2`

4. 흐름

1. 값을 정리합니다. 검색어는 공백을 지우고 100자까지, 정렬은 3가지 중 하나, 페이지는 1 이상의 정수로 만듭니다.
2. `GET /projects?query=&sort=&page=&size=10`을 보냅니다.
3. 실패하면 401은 로그인으로, 그 밖은 오류 화면으로 옮기고 `null`을 돌려줍니다.
4. 성공하면 `current`에 조건을 기억하고, 기본값이 아닌 것만 주소에 남긴 뒤(`#…`는 그대로) 응답을 돌려줍니다.

5. 왜 이렇게

선택	이유
값을 먼저 정리	주소는 사용자가 손으로 고칠 수 있습니다 (`?page=abc`). 서버에 보내서 422를 받는 것보다 미리 기본값으로 바꾸는 편이 낫습니다. 사용자에게는 그냥 첫 페이지가 보입니다
오류는 `loadMe`(22)와 같은 규칙	템플릿에는 "500 이상 → 500"만 적혀 있었지만, 같은 모양의 페이지 데이터 API이므로 401, 503, 네트워크 오류도 같은 방식으로 처리합니다 (2026-10-06 결정: "본 데이터 API 오류 → 오류 화면")
그 밖의 4xx는 400 화면	값을 정리하므로 일어나지 않아야 하지만, 생긴다면 빈 화면보다 "잘못된 요청" 화면이 낫습니다
실패하면 `null`을 돌려줌	부르는 쪽(38 등)이 `if (!data) return;` 한 줄로 멈출 수 있습니다. 오류 화면으로 옮겨 가는 중에 그리기를 하지 않게 합니다
주소에는 **기본값을 뺀** 것만 남김	`myProject.html?query=&sort=updated&page=1`보다 `myProject.html`이 깔끔합니다. 읽을 때(38)는 값이 없으면 기본값을 씁니다
`replaceState`를 씀 (`pushState`가 아님)	검색할 때마다 뒤로 가기 기록이 쌓이지 않게 합니다
주소 갱신은 **성공한 뒤에**	실패해서 오류 화면으로 가는데 주소만 바뀌는 일이 없게 합니다
`current`를 밖으로 내보냄	페이지 넘김, 새 프로젝트, 삭제 뒤에 "같은 조건으로 다시 받기"를 할 때 씁니다

선택	이유
옛 응답 버리기 (`loadTicket`, 33에서 추가)	빠르게 검색하면 먼저 보낸 요청이 나중에 도착해서 새 결과를 덮어쓸 수 있습니다. 그래서 요청마다 번호를 붙이고 마지막 것만 씁니다 (33 §5)
`ERROR_PAGES`를 userinfo.js와 따로 둠	지금은 두 곳뿐이라 그대로 두었습니다. 세 번째 페이지(P1-17 myPage)에서도 쓰게 되면 `core/`로 옮기는 것이 좋습니다

6. 시험 (실제 myProject.html, 실제 서버, 헤드리스 Chrome, 14개)

시험 계정 `sitetest.p114@example.com`으로 가입하고 로그인한 뒤 프로젝트 3개(Beta → alpha → Gamma 순서로 만듦)를 만들고, 페이지 안에서 함수를 불렀습니다. 오류 시험은 `window.fetch`를 가짜로 바꿔서 서버 응답을 흉내 냈습니다. 끝난 뒤 계정과 프로젝트를 지웠습니다.

#	부른 값	결과와 주소
01	기본	`Gamma, alpha, Beta` (최근 수정순), total 3, 주소 `myProject.html`
02	`query: ' gam ', sort: 'title'`	`Gamma`, current `{query:'gam', sort:'title', page:1}`, `?query=gam&sort=title`
03	`sort: 'random', page: 'abc'`	updated와 1로 고쳐서 3개를 받고, 주소는 깨끗합니다
04	`page: 0`, `page: -3`	둘 다 1이 됩니다
05	`page: 2` (10개씩이라 없는 쪽)	빈 목록, total 3, `?page=2`
06	`query: 'a&b%'`	0개, `?query=a%26b%25` (특수 문자가 주소를 깨지 않습니다)
07	150자 검색어	100자로 잘라 보내서 422 없이 0개를 받습니다
08	주소 `?query=old&page=9#top`에서 `sort: 'created'`	`?sort=created#top`이 됩니다. 옛 값은 지우고 `#top`은 남깁니다
09	가짜 500	`errors/500.html`
10	가짜 503	`errors/503.html`
11	가짜 502	`errors/500.html`
12	가짜 422	`errors/400.html`
13	네트워크 끊김 (fetch 실패)	`errors/503.html`
14	로그아웃한 뒤	`home.html#login?next=%2FmyProject.html`

JS 31 — `renderProjects()` (+ `buildRow`)

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`, (CSS) `static/css/pages/myproject.css`
작업	P1-14 내 프로젝트 화면: 목록 그리기 (PRD 01 FR-123, FR-125, FR-130)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	30 `loadProjects()` · 32 `renderPager()`

1. 하는 일

`loadProjects`(30)가 받은 목록을 **화면의 줄로 그립니다**. HTML의 예시 줄 7개를 지우고, `<template id="project-row-template">`을 프로젝트 수만큼 복사해서 채웁니다.

칸	넣는 것	예
제목	제목과, 아직 열지 않았으면 `Not opened` 표시	`Lo-fi Sketch [Not opened]`
설명	설명이며, 없으면 `No description`	
BPM, Sig, Key	`Number(bpm)`, `7/8`, 키가 없으면 `–`	`'128.50'` → `128.5`, `'120.00'` → `120`
Members	점을 최대 3개 그리고, OWNER가 아니면 역할을 씁니다	5명, EDITOR → ●●● `Editor`
Edited	`formatRelativeTime(updated_at)`(29)과 휴대폰용 `data-meta`	`12 minutes ago`, meta `128 BPM · A minor ·`
… 메뉴	OWNER가 아니면 Delete를 잠그고 `Owner only`를 붙이며, VIEWER는 Edit도 잠급니다	
링크	제목과 Open이 `studioNx.html?projectId=번호`를 가리킵니다	

목록	화면
1개 이상	개수 `2 projects`(1개면 `1 project`), 제목 줄, 목록
0개, 검색어 없음	`No projects yet. Create your first project`
0개, 검색어 있음	`No projects match your search.`

개수는 **이 페이지의 줄 수가 아니라 전체 개수**(`total`)입니다. 23개 중 1쪽을 보고 있으면 `23 projects`입니다.

2. 언제 쓰나

호출하는 곳	화면
페이지 시작, 검색, 페이지 넘김, 새 프로젝트, 삭제 뒤 (32, 33, 35, 37, 38)	항상 `loadProjects` 다음에 부릅니다: `const data = await loadProjects(…); if (data) renderProjects(data);`

`<template>`은 화면에 보이지 않는 HTML 틀입니다. `template.content` 안의 요소를 `cloneNode(true)`로 복사하면 같은 모양의 새 줄이 생깁니다. 줄 모양은 HTML이 정하고, JS는 값만 채웁니다 (나중에 React로 옮길 때도 틀을 그대로 컴포넌트로 바꿀 수 있습니다).

3. 코드

import (`myproject.js` 맨 위: 30번과 같고, 이번에 추가한 것은 없습니다)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio } from '../core/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

이 함수는 import를 쓰지 않습니다. 같은 파일의 `current`(30)와 `formatRelativeTime`(29)만 씁니다.

함수 (줄 하나를 만드는 부분은 `buildRow`로 나눴습니다. 파일 밖에서는 쓰지 않습니다)

```
export function renderProjects(data) {
  const list = document.getElementById('project-list');
  const template = document.getElementById('project-row-template');
  const count = document.getElementById('project-count');
  const label = document.querySelector('.pj-list .pj-row--label');
  const empty = document.getElementById('project-empty');
  const noResult = document.getElementById('project-no-result');
  if (!list || !template) return;

  // 1. 개수 · 빈 목록 안내
  if (count) count.textContent = data.total === 1 ? '1 project' : data.total + ' projects';
  const isEmpty = data.items.length === 0;
  list.hidden = isEmpty;
  if (label) label.hidden = isEmpty;
  if (empty) empty.hidden = !(isEmpty && !current.query);
  if (noResult) noResult.hidden = !(isEmpty && current.query);

  // 2. 줄 그리기 — 예시 줄을 지우고 틀을 복사해 채움
  list.textContent = '';
  for (let i = 0; i < data.items.length; i++) {
    list.appendChild(buildRow(template, data.items[i]));
  }
}

/** 프로젝트 하나 → 목록 한 줄 <li> (renderProjects 가 씀) */
function buildRow(template, item) {
  const row = template.content.firstElementChild.cloneNode(true);
  const field = function (name) { return row.querySelector('[data-field="' + name + '"]'); };
  const href = 'studioNx.html?projectId=' + item.project_id;
  const bpm = String(Number(item.bpm));             // '128.50' → '128.5', '120.00' → '120'
  const key = item.music_key ? item.music_key : '—';
  const isOwner = item.my_role === 'OWNER';

  row.dataset.projectId = item.project_id;
  row.dataset.title = item.title;                                        // 34 가 수정 칸을 채울 때 씀
  row.dataset.description = item.description ? item.description : '';
  row.querySelector('.pj-row__head').href = href;
  row.querySelector('.pj-row__act a').href = href;
```

```
  // 제목 · 설명 · Not opened
  const title = row.querySelector('.pj-row__head b');
  title.textContent = item.title;
  if (!item.opened) {
    const badge = document.createElement('span');
    badge.className = 'pj-badge';
    badge.textContent = 'Not opened';
    title.appendChild(document.createTextNode(' '));
    title.appendChild(badge);
  }
  row.querySelector('.pj-row__head > span').textContent = item.description ? item.description : 'No desc
ription';

  // BPM · 박자 · 키 · 언제 (모바일 한 줄 정보는 data-meta)
  field('bpm').textContent = bpm;
  field('sig').textContent = item.time_sig_num + '/' + item.time_sig_den;
  field('key').textContent = key;
  const when = field('when');
  when.textContent = formatRelativeTime(item.updated_at);
  when.dataset.meta = bpm + ' BPM · ' + (item.music_key ? item.music_key + ' · ' : '');

  // 참여자 점 (최대 3개) + 내 역할 (OWNER 가 아니면)
  const members = field('members');
  for (let i = 0; i < Math.min(item.member_count, 3); i++) {
    members.appendChild(document.createElement('i'));
  }
  let membersLabel = item.member_count === 1 ? '1 member' : item.member_count + ' members';
  if (!isOwner) {
    const roleName = item.my_role.charAt(0) + item.my_role.slice(1).toLowerCase();   // 'EDITOR' → 'Edit
or'
    const role = document.createElement('span');
    role.className = 'pj-role';
    role.textContent = roleName;
    members.appendChild(role);
    membersLabel += ', your role: ' + roleName.toLowerCase();
  }
  members.setAttribute('aria-label', membersLabel);

  // ··· 메뉴 — 삭제는 OWNER 만, 수정은 OWNER · EDITOR 만
  row.querySelector('.pj-more').setAttribute('aria-label', 'More actions for ' + item.title);
  if (!isOwner) {
    const deleteButton = row.querySelector('[data-action="delete"]');
    deleteButton.disabled = true;
    deleteButton.title = 'Only the owner can delete';
    const small = document.createElement('small');
    small.textContent = 'Owner only';
    deleteButton.appendChild(document.createTextNode(' '));
    deleteButton.appendChild(small);
  }
  if (item.my_role === 'VIEWER') {
    const editButton = row.querySelector('[data-action="edit"]');
    editButton.disabled = true;
    editButton.title = 'Viewers can’t edit';
  }
  return row;
}
```

CSS (`myproject.css`, 이번에 추가)

```
/* JS 가 hidden 으로 숨기는 것 (목록 0개 · 페이지 1개) — display: grid · flex 가 hidden 을 이기지 않게 (P1-14) */
.pj-list [hidden], .pj-pager[hidden] { display: none; }
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
list.replaceChildren(...data.items.map((item) => buildRow(template, item)));
const field = (name) => row.querySelector(`[data-field="${name}"]`);
members.append(...Array.from({ length: Math.min(item.member_count, 3) }, () => document.createElement('i')));
title.append(' ', badge);
```

최신 문법	실제 코드	뜻
`list.replaceChildren(...줄들)`	`list.textContent = ''` + `for`로 `appendChild`	안을 비우고 새로 채웁니다
`(name) => …` (화살표 함수)	`function (name) { return …; }`	짧은 함수입니다
`Array.from({ length: n }, () => …)`	`for (let i = 0; i < n; i++)`	n번 만듭니다
`el.append(' ', badge)`	`appendChild(createTextNode(' '))` + `appendChild(badge)`	글자와 요소를 한 번에 붙입니다

문법 풀이

import 문법	뜻
(이 함수는 import 없음)	같은 파일의 `current`와 `formatRelativeTime`을 그대로 부릅니다

식	뜻	예
`template.content.firstElementChild.cloneNode(true)`	틀 안의 첫 요소(`<li>`)를 **안쪽까지 통째로** 복사합니다	`true`가 없으면 `<li>` 껍데기만 복사됩니다
`el.textContent = 값`	글자로만 넣습니다. `<img …>`도 태그가 아니라 글자로 보입니다	제목이 `<img src=x onerror=…>`이면 그 글자 그대로 보입니다
`list.textContent = ''`	안쪽을 모두 지웁니다 (HTML 예시 줄 포함)	
`row.dataset.projectId = 5`	`data-project-id="5"` 속성입니다 (34, 36, 37이 읽습니다)	
`String(Number('128.50'))`	문자열을 숫자로, 다시 문자열로 바꾸면 끝의 0이 빠집니다	`'128.5'`, `'120'`
`item.my_role.charAt(0) + item.my_role.slice(1).toLowerCase()`	첫 글자만 대문자로 만듭니다	`'EDITOR'` → `'Editor'`
`'.pj-row__head > span'`	`<a>` **바로 아래**의 span입니다 (제목 안의 `Not opened` span은 아닙니다)	설명 칸
`!(isEmpty && !current.query)`	"비었고 검색어도 없음"이 **아니면** 숨깁니다	

4. 흐름

1. 개수 글자와 빈 목록 안내(검색어 유무에 따라 둘 중 하나)를 정하고, 목록과 제목 줄을 보이거나 숨깁니다.

2. 목록을 비운 뒤 프로젝트마다 `buildRow`를 부릅니다. `buildRow`는 틀을 복사하고, 번호와 링크, 제목(+ Not opened)과 설명, BPM, 박자, 키, 수정 시각, 멤버 점과 역할을 채운 뒤 … 메뉴를 잠급니다.

5. 왜 이렇게

선택	이유
모든 글자를 `textContent`로 넣음	제목과 설명은 사용자가 입력한 글자입니다. `innerHTML`을 쓰면 `<img onerror=…>` 같은 제목이 **실행됩니다** (XSS). 시험에서 그대로 글자로 보이고 이미지가 0개임을 확인했습니다
줄 만들기를 `buildRow`로 나눔	칸이 많아서 한 함수에 다 넣으면 길어집니다. 36번(수정 뒤 그 줄만 다시 그리기)에서도 씁니다
`Number(bpm)`	서버가 소수를 문자열 `'128.50'`으로 보냅니다 (P1-13에서 확인). 화면 예시처럼 `128`, `128.5`로 보여 줍니다
개수는 `total`	페이지당 10개라서 `items.length`는 최대 10입니다. 사용자가 알고 싶은 것은 전체 개수입니다
빈 목록 안내를 두 가지로	처음 쓰는 사용자에게는 "만들기" 버튼을, 검색 결과가 없을 때는 "검색어를 바꾸라"는 안내를 보여 줍니다 (템플릿대로)
VIEWER는 Edit도 잠금	템플릿에는 Delete만 있었지만, 서버가 VIEWER의 수정을 403으로 막습니다(P1-13 60). 누르고 실패하는 것보다 미리 잠그는 편이 낫습니다. Phase 1에서는 OWNER만 생기므로 지금은 보이지 않습니다
멤버 점은 최대 3개	화면 예시와 템플릿을 따랐습니다. 정확한 수는 `aria-label`(`5 members, your role: viewer`)로 화면 읽기 프로그램에 알려 줍니다
`data-title`, `data-description` (34에서 추가)	34번이 수정 칸을 채울 때, 화면 글자(`Not opened`, `No description`이 섞임) 대신 이 값을 읽습니다
CSS에 `[hidden]` 규칙 추가	`.pj-row`가 `display: grid`라서 `hidden` 속성을 이기는 바람에 제목 줄이 숨지 않았습니다. 규칙이 없으면 `.pj-row`는 hidden이어도 `grid`로 계산된다는 것을 시험으로 확인했습니다. 32번 페이지 넘김(`.pj-pager`는 flex)도 같은 문제라서 함께 고쳤습니다

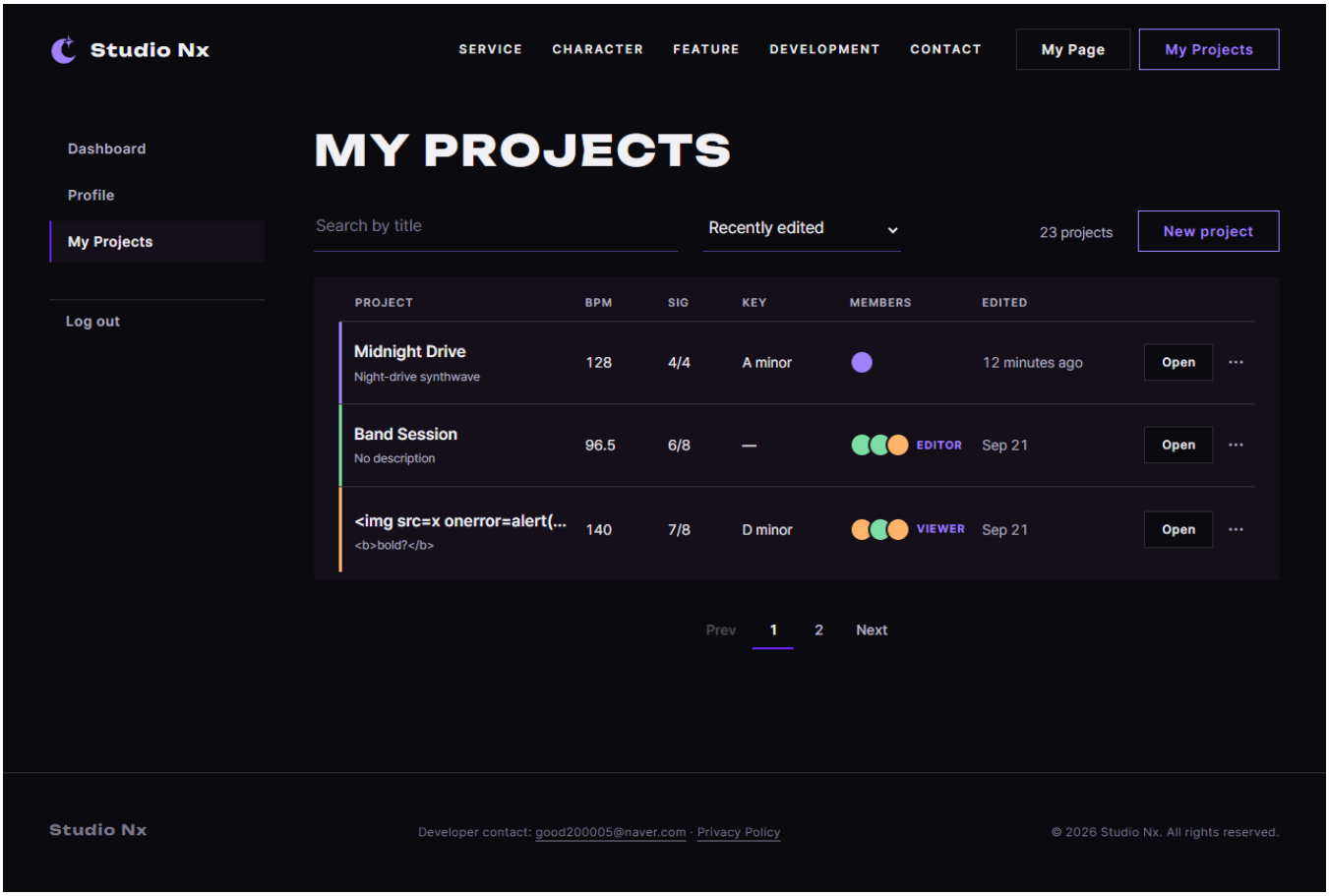
6. 시험 (실제 myProject.html, 실제 서버, 헤드리스 Chrome)

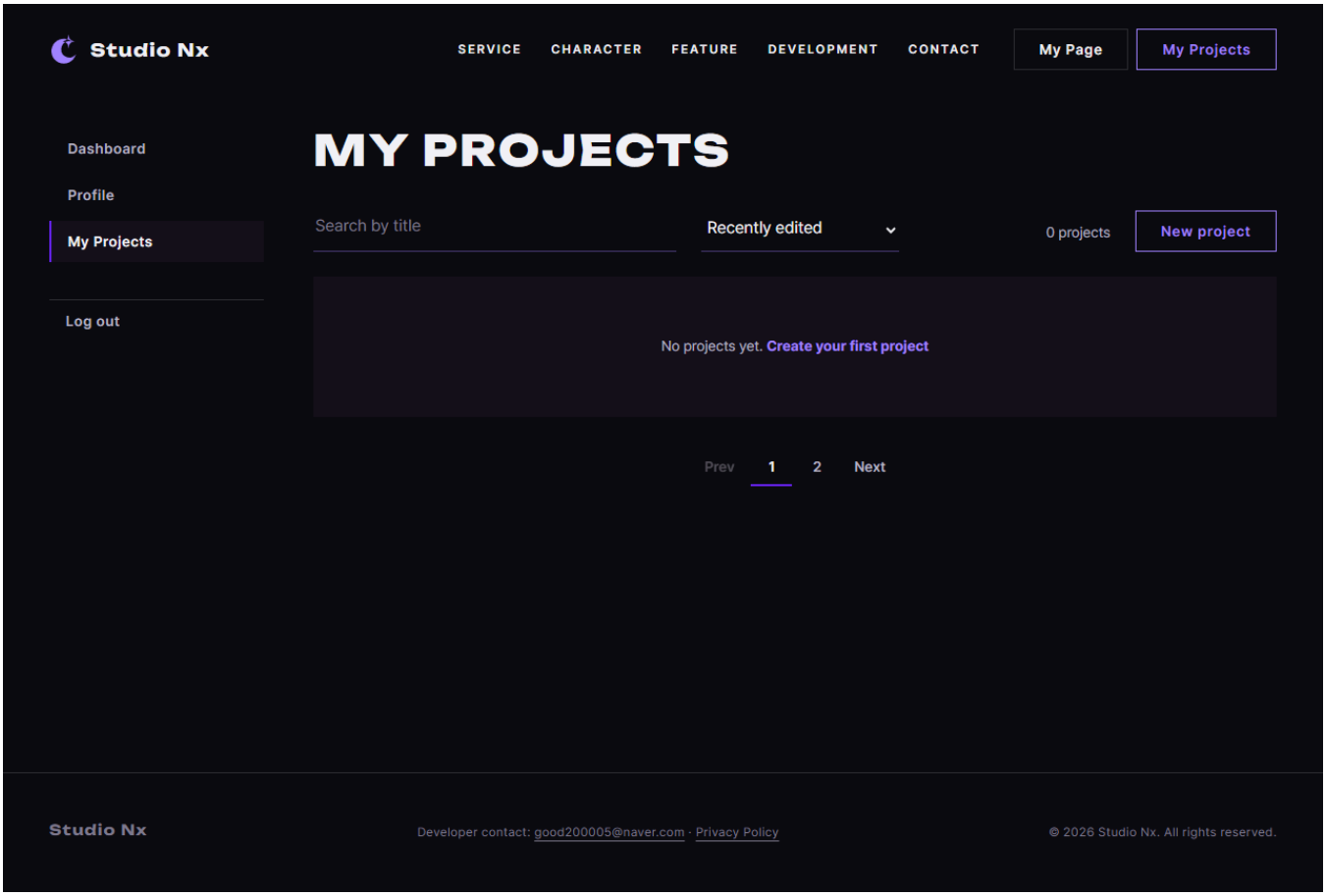
시험 계정으로 프로젝트 2개를 만들어 실제 목록을 그렸고, 역할, 멤버, 키, XSS는 가짜 응답으로 그렸습니다. 보이는지는 **계산된 CSS**(`getComputedStyle(…).display`)로 확인했습니다. 끝난 뒤 계정과 프로젝트를 지웠습니다.

#	상황	결과
01	실제 서버, 2개	`2 projects`이고, 최근 것이 먼저 나옵니다. `Lo-fi Sketch`는 bpm `128.5`, `7/8`, 키 `–`, `Not opened`이고, `First Idea`는 설명 `No description`, bpm `120`, `Just now`이며, 링크는 `studioNx.html?projectId=…`입니다
02	가짜 3개 (total 23)	`23 projects`입니다. OWNER는 역할 표시가 없고 Delete가 열려 있으며 `12 minutes ago`, meta `128 BPM · A minor ·` 입니다. EDITOR 3명은 점 3개, `Editor`, Delete 잠금 `Owner only`, Edit 열림, `Sep 21`입니다. VIEWER 5명은 점 3개, `5 members, your role: viewer`, Edit와 Delete 모두 잠금입니다
02	제목 `<img src=x onerror=alert(1)>`, 설명 `<b>bold?</b>`	글자 그대로 나오고, 목록 안의 이미지는 0개입니다
03	0개, 검색어 없음	`0 projects`이고, `No projects yet`이 보이며, 목록과 제목 줄은 숨겨집니다
04	0개, 검색어 `zzz` (실제)	`No projects match your search.`가 보입니다

#	상황	결과
05	다시 1개	`1 project` 이고, 목록과 제목 줄이 다시 보입니다

캡처




Studio Nx
SERVICE
CHARACTER
FEATURE
DEVELOPMENT
CONTACT
My Page
My Projects
Dashboard
Profile
My Projects
Log out
MY PROJECTS
Search by title
Recently edited
0 projects
New project
No projects yet. Create your first project
Prev
1
2
Next
Studio Nx
Developer contact: good200005@naver.com · Privacy Policy
© 2026 Studio Nx. All rights reserved.

JS 32 — `renderPager()`

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면: 페이지 넘김 (PRD 01 FR-123)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	31 `renderProjects()` · 33 `initSearch()`

1. 하는 일

목록 아래의 페이지 넘김 줄(`#project-pager`)을 **전체 개수에 맞게 다시 그리고**, 번호를 누르면 그 페이지를 받아서 목록과 이 줄을 다시 그립니다 (페이지는 새로 열리지 않습니다).

전체 (10개씩)	지금 페이지	화면 (별표 = 지금, (x) = 막힘)
0개, 10개	—	줄을 숨깁니다
11개 (2쪽)	1	`Prev(x) 1* 2 Next`
3쪽	3	`Prev 1 2 3* Next(x)`
7쪽	4	`Prev 1 2 3 4* 5 6 7 Next` (7쪽까지는 모두 보여 줍니다)
20쪽	10	`Prev 1 … 9 10* 11 … 20 Next`
20쪽	4	`Prev 1 2 3 4* 5 … 20 Next` (…가 한 쪽만 가리면 번호로 보여 줍니다)
20쪽	20	`Prev 1 … 19 20* Next(x)`

링크 주소에도 검색어와 정렬이 들어갑니다. 검색어가 `lo fi`이고 이름순이면 2쪽 링크는 `?query=lo+fi&sort=title&page=2`입니다. 그래서 **새 탭으로 열어도** 같은 목록이 나옵니다 (그 주소를 읽는 일은 38번이 합니다).

2. 언제 쓰나

호출하는 곳	화면
`renderProjects` 다음마다 (페이지 시작, 검색, 새 프로젝트, 삭제: 33, 35, 37, 38)	`renderPager(data.total, data.page, data.size)`
이 함수 자신	번호를 누른 뒤 새 페이지 기준으로 다시 그립니다

3. 코드

import (`myproject.js` 맨 위: 30번과 같고, 이번에 추가한 것은 없습니다)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio } from '../core/studio.js';
```

```
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

이 함수는 import를 쓰지 않습니다. 같은 파일의 `PAGE_SIZE`, `current`, `loadProjects`(30), `renderProjects`(31)를 씁니다.

함수 (링크 주소 만들기는 `pageHref`로 나눴습니다)

```
export function renderPager(total, page, size = PAGE_SIZE) {
  const pager = document.getElementById('project-pager');
  if (!pager) return;
  const pages = Math.max(1, Math.ceil(total / size));
  pager.textContent = '';
  pager.hidden = pages <= 1;
  if (pages <= 1) return;

  // 버튼 하나 — 갈 수 없으면 href 없이 aria-disabled
  const addButton = function (text, target, label) {
    const button = document.createElement('a');
    button.className = 'pj-pager__btn';
    button.textContent = text;
    if (label) button.setAttribute('aria-label', label);
    if (target < 1 || target > pages) {
      button.setAttribute('aria-disabled', 'true');
    } else {
      button.href = pageHref(target);
      button.dataset.page = target;
      if (target === page && !label) button.setAttribute('aria-current', 'page');
    }
    pager.appendChild(button);
  };
  const addGap = function () {
    const gap = document.createElement('span');
    gap.className = 'pj-pager__btn';
    gap.setAttribute('aria-hidden', 'true');
    gap.textContent = '…';
    pager.appendChild(gap);
  };

  addButton('Prev', page - 1, 'Previous page');
  for (let n = 1; n <= pages; n++) {
    // 7쪽 이하는 전부, 넘으면 처음 · 끝 · 지금 앞뒤 하나만 (사이는 …)
    const near = n === 1 || n === pages || Math.abs(n - page) <= 1;
    if (pages <= 7 || near) {
      addButton(String(n), n);
    } else if (n === page - 2 || n === page + 2) {
      // … 가 한 쪽만 가리면 (2 · 끝에서 둘째) 그냥 그 번호를 보임
      if (n === 2 || n === pages - 1) addButton(String(n), n);
      else addGap();
    }
  }
  addButton('Next', page + 1, 'Next page');

  // 클릭 → 같은 검색어 · 정렬로 그 페이지를 받아 다시 그림 (페이지는 바뀌지 않음)
  pager.onclick = async function (event) {
    const button = event.target.closest('a[data-page]');
```

```
    if (!button) return;
    event.preventDefault();
    const data = await refreshList({ query: current.query, sort: current.sort, page: button.dataset.page
});
    if (data) window.scrollTo(0, 0);
  };
}

/** 페이지 링크 주소 — 지금 검색어 · 정렬 + 그 페이지 (loadProjects 가 주소창에 남기는 모양과 같음) */
function pageHref(page) {
  const params = new URLSearchParams();
  if (current.query) params.set('query', current.query);
  if (current.sort !== 'updated') params.set('sort', current.sort);
  if (page !== 1) params.set('page', String(page));
  const search = params.toString();
  return search ? '?' + search : location.pathname;
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
const addButton = (text, target, label) => { … };
pager.addEventListener('click', async (event) => { … }, { signal });   // 다시 그릴 때 AbortController 로 옛 것 떼기
const { query, sort } = current;
const data = await loadProjects({ query, sort, page: button.dataset.page });
```

최신 문법	실제 코드	뜻
`(a, b) => { … }`	`function (a, b) { … }`	함수입니다
`addEventListener(…, { signal })`	`pager.onclick = …`	다시 그려도 클릭 처리가 하나만 남게 합니다
`const { query, sort } = current` + `{ query, sort }`	`{ query: current.query, sort: current.sort }`	칸을 꺼내고 줄임 속성으로 씁니다

문법 풀이

import 문법	뜻
(이 함수는 import 없음)	같은 파일의 함수와 값을 그대로 부릅니다

식	뜻	예
`Math.ceil(total / size)`	올림입니다. 남는 것이 있으면 한 쪽을 더합니다	11 / 10 = 1.1 → 2
`Math.max(1, …)`	0쪽이 되지 않게 합니다	0개 → 1쪽 (숨김)
`Math.abs(n - page) <= 1`	지금 페이지와 1 이하로 떨어진 번호입니다	지금이 10이면 9, 10, 11
`button.dataset.page = target`	`data-page="2"` 입니다. 누를 수 있는 번호에만 붙입니다	Prev가 막혔으면 붙이지 않습니다
`aria-current="page"`	화면 읽기 프로그램과 CSS가 "지금 페이지"로 알아봅니다 (보라색 밑줄)	
`aria-disabled="true"`	막힌 버튼입니다. CSS가 흐리게 만들고 눌리지 않게 합니다	
`pager.onclick = 함수`	클릭 처리를 **바꿔 끼웁니다**. 몇 번을 다시 그려도 하나만 남습니다	`addEventListener`는 부를 때마다 하나씩 쌓입니다

식	뜻	예
`event.target.closest('a[data-page]')`	누른 곳에서 위로 올라가며 번호 버튼을 찾습니다	`…`나 막힌 버튼은 null이므로 무시합니다
`event.preventDefault()`	링크의 기본 동작(페이지 새로 열기)을 막고, JS로 받아서 그립니다	
`window.scrollTo(0, 0)`	맨 위로 올립니다. 페이지 줄은 목록 아래에 있기 때문입니다	

4. 흐름

1. 페이지 수를 계산합니다. 1쪽 이하면 줄을 숨기고 끝냅니다.
2. `Prev`, 번호들(7쪽 이하면 모두, 넘으면 처음, 끝, 지금 앞뒤와 `…`), `Next`를 차례로 만듭니다.
3. 번호를 누르면 링크의 기본 동작을 막고, 같은 검색어와 정렬로 그 페이지를 받은 뒤(30), 목록(31)과 이 줄을 다시 그리고 맨 위로 올립니다.

5. 왜 이렇게

선택	이유
7쪽이 넘으면 줄임	프로젝트가 많으면 버튼이 한 줄을 넘칩니다. 처음, 끝, 지금 근처만 있어도 어디로든 두세 번 안에 갈 수 있습니다
`…`가 한 쪽만 가리면 번호로 보여 줌	`1 … 3`의 `…`는 2 하나뿐이므로, 그냥 `1 2 3`이 더 짧고 정확합니다 (처음 만든 뒤 시험에서 보고 고쳤습니다)
링크에 실제 주소(`href`)를 넣음	마우스 가운데 버튼이나 새 탭으로 열기가 됩니다. 클릭은 JS가 가로채므로 페이지를 새로 열지 않습니다
막힌 Prev, Next에는 `href`가 없음	누를 곳이 아니기 때문입니다 (화면 예시와 같습니다). CSS가 흐리게 만들고 클릭되지 않게 합니다
`onclick` 하나만 씀	이 함수는 페이지를 넘길 때마다 다시 불립니다. `addEventListener`를 쓰면 한 번 누를 때 요청이 여러 번 갑니다. 시험에서 세 번 다시 그린 뒤 한 번 눌러서 요청이 **1번**임을 확인했습니다
클릭 뒤 처리는 `refreshList`로 묶음 (33에서)	받기 → 목록 → 페이지 줄의 세 줄이 검색, 새 프로젝트, 삭제와 같기 때문입니다
지금 페이지도 링크로 둠	화면 예시와 같고, 다시 누르면 새로 받습니다

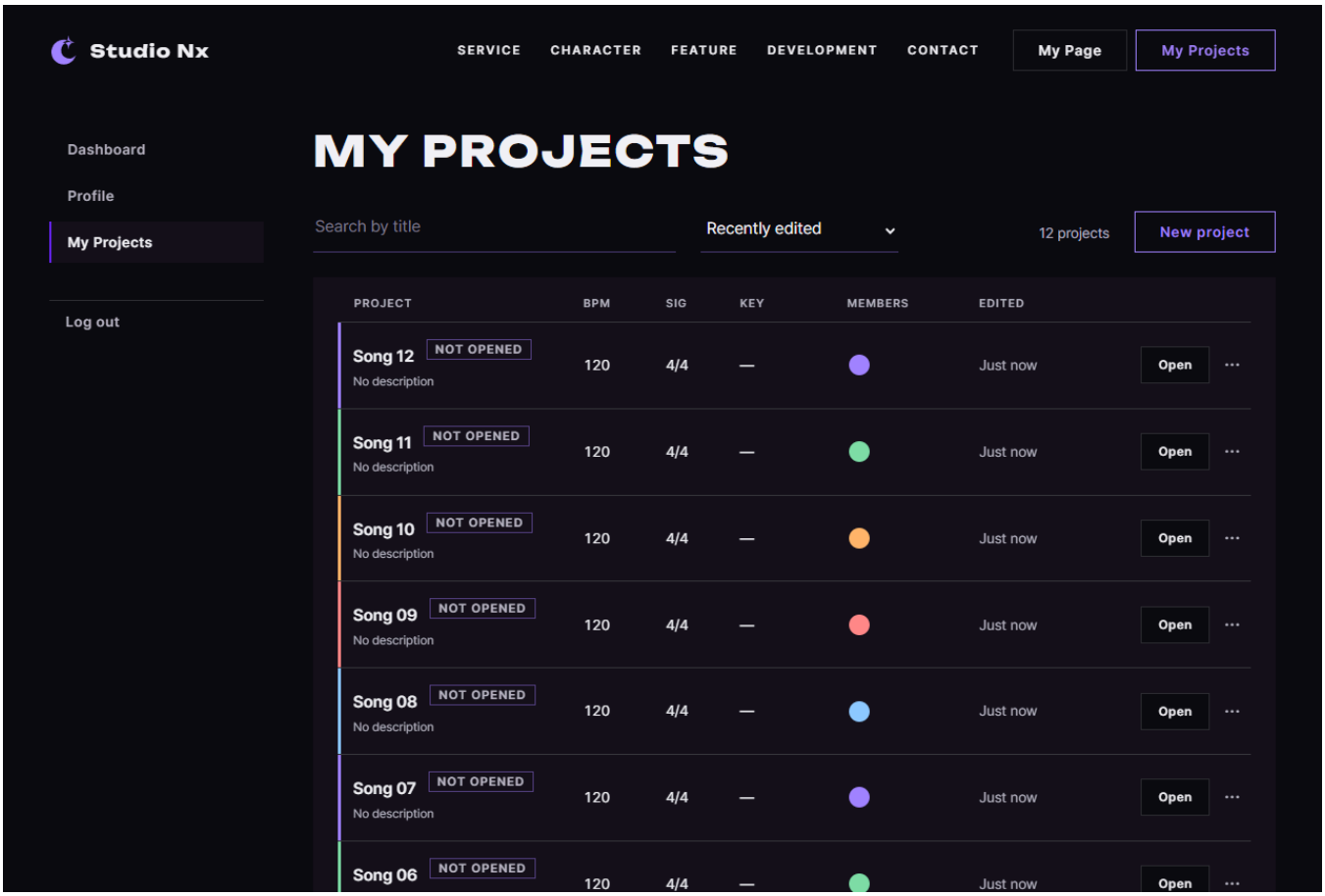
6. 시험 (실제 myProject.html, 헤드리스 Chrome, 10개)

#	상황	결과
01	전체 0개, 10개, 11개	숨김, 숨김, `Prev(x) 1* 2 Next` (숨김은 계산된 CSS로 확인했습니다)
02, 03	3쪽 중 1쪽, 3쪽	`Prev(x) 1* 2 3 Next`, `Prev 1 2 3* Next(x)`
04	20쪽 중 10쪽	`Prev 1 … 9 10* 11 … 20 Next`
05	20쪽 중 1, 3, 4, 20쪽	`1* 2 … 20`, `1 2 3* 4 … 20`, `1 2 3 4* 5 … 20`, `1 … 19 20*`
06	8쪽 중 4쪽, 7쪽 중 4쪽	`1 2 3 4* 5 … 8`, `1 2 3 4* 5 6 7`
07	검색어 `lo fi`, 이름순	링크가 `?query=lo+fi&sort=title&page=2` 등이 됩니다. 기본 조건이면 1쪽 링크는 `/myProject.html`입니다

#	상황	결과
08	실제 프로젝트 12개에서 2를 누름	목록 2줄, `Prev 1 2* Next(x)`, 주소 `?page=2`이고, 요청은 **1번**입니다 (세 번 다시 그린 뒤)
08	이어서 Prev를 누름	목록 10줄, `Prev(x) 1* 2 Next`, 주소 `myProject.html`
09	막힌 Prev를 누름	요청 0번이고, 주소는 그대로입니다

처음 만든 것은 20쪽 중 4쪽에서 `1 … 3 4 5`로 나와서 `…`가 2 하나만 가렸습니다. 번호로 보이게 고친 뒤 01~09를 다시 돌려서 모두 통과했습니다.

캡처: 2쪽



JS 33 — `initSearch()` (+ `refreshList`)

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면: 검색과 정렬 (PRD 01 FR-123)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	32 `renderPager()` · 34 `initRowMenus()`

1. 하는 일

목록 위의 검색 칸이나 정렬 칸이 바뀌면 **1쪽부터 목록을 다시 받아서 그립니다**.

사용자가 한 일	언제 요청하나	결과 (주소)
검색 칸에 `alp`를 입력	입력을 **멈추고 0.3초 뒤에 한 번**	`alpine, Alpha Song` (`?query=alp`)
공백을 하나 더 입력	요청하지 않습니다 (검색어가 같음)	그대로
정렬을 Name으로 바꿈	**바로**	`Alpha Song, alpine` (`?query=alp&sort=title`)
검색 칸을 비움	0.3초 뒤	전체 4개 (`?sort=title`)
`gam`을 입력하고 바로 Enter	**바로, 한 번만** (기다리던 요청은 취소)	`Gamma Loop`

함께 만든 것

- `refreshList(options)`: "받기(30) → 목록(31) → 페이지 줄(32)"의 세 줄을 한 함수로 묶었습니다. 32번 페이지 넘김도 이것을 쓰도록 바꿨고, 35번과 37번도 씁니다.
- **30번 `loadProjects`에 두 줄 추가: 늦게 도착한 옛 응답 버리기.** 아래 §5를 참고하십시오.

2. 언제 쓰나

호출하는 곳	화면
페이지 시작 (`myproject.js` 맨 아래, 이미 있음)	`initSearch();`로 검색 칸과 정렬 칸에 이벤트 처리를 한 번 걸어 둡니다

디바운스(debounce)는 글자를 칠 때마다 요청하지 않고, **입력이 멈춘 뒤** 정해진 시간이 지나면 한 번만 요청하는 방법입니다. 칠 때마다 이전 타이머를 지우고(`clearTimeout`) 새로 걸기(`setTimeout`) 때문에 마지막 타이머만 남습니다. `alp`를 0.06초 간격으로 치면 요청은 3번이 아니라 1번만 갑니다.

3. 코드

import (`myproject.js` 맨 위: 30번과 같고, 이번에 추가한 것은 없습니다)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio } from '../core/studio.js';
```

```
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

이 함수는 import를 쓰지 않습니다. 같은 파일의 `current`, `loadProjects`, `renderProjects`, `renderPager`를 씁니다.

상수 (이번에 추가)

```
/** 검색 칸 입력을 멈춘 뒤 기다리는 시간 (ms) */
const SEARCH_DELAY = 300;

/** loadProjects 요청 번호 — 늦게 도착한 옛 응답을 버리려고 */
let loadTicket = 0;
```

함수

```
export function initSearch() {
  const form = document.getElementById('project-search');
  const input = document.getElementById('project-query');
  const sort = document.getElementById('project-sort');
  if (!form || !input || !sort) return;
  let timer = null;

  const search = function () {
    clearTimeout(timer);
    refreshList({ query: input.value, sort: sort.value, page: 1 });
  };

  input.addEventListener('input', function () {
    clearTimeout(timer);
    if (input.value.trim() === current.query) return;   // 공백만 더 쳤거나 원래대로 되돌림
    timer = setTimeout(search, SEARCH_DELAY);
  });
  sort.addEventListener('change', search);
  form.addEventListener('submit', function (event) {
    event.preventDefault();   // 페이지를 새로 열지 않음
    search();
  });
}

/** 목록 다시 받아 그리기 — loadProjects → renderProjects → renderPager (실패 · 옛 응답이면 아무것도 안
함) */
async function refreshList(options) {
  const data = await loadProjects(options);
  if (!data) return null;
  renderProjects(data);
  renderPager(data.total, data.page, data.size);
  return data;
}
```

30번 `loadProjects`에 추가한 부분

```
  const ticket = ++loadTicket;
  let data;
  try {
    data = await api.get('/projects?' + params.toString());
```

```
  } catch (err) {
    if (ticket !== loadTicket) return null;   // 더 새 요청이 있으면 이 실패는 무시
    …
  }
  if (ticket !== loadTicket) return null;   // 그사이 더 새 요청을 보냄 → 이 응답은 버림 (33)
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
input.addEventListener('input', () => { … });
form.addEventListener('submit', (event) => { event.preventDefault(); search(); });
```

최신 문법	실제 코드	뜻
`() => { … }`	`function () { … }`	함수입니다

문법 풀이

import 문법	뜻
(이 함수는 import 없음)	같은 파일의 함수를 부릅니다

식	뜻	예
`setTimeout(search, SEARCH_DELAY)`	0.3초 뒤에 `search`를 한 번 부르고, 타이머 번호를 돌려줍니다	
`clearTimeout(timer)`	아직 불리지 않은 타이머를 취소합니다 (이미 지났거나 null이면 아무 일도 없습니다)	다음 글자를 치면 앞의 타이머를 취소합니다
`input.value.trim() === current.query`	지금 목록의 검색어와 같으면 요청할 필요가 없습니다	`'alp '`와 `'alp'`는 같습니다
`'input'`, `'change'`, `'submit'` 이벤트	차례대로 글자가 바뀔 때마다, select 값을 고른 뒤, Enter(폼 보내기)	
`event.preventDefault()`	폼의 기본 동작(주소에 `?query=`를 붙여 페이지를 새로 열기)을 막습니다	
`++loadTicket`	1을 늘린 **뒤의** 값입니다. 요청마다 새 번호가 됩니다	1, 2, 3 …
`ticket !== loadTicket`	내가 보낸 뒤에 다른 요청이 또 나갔는지 확인합니다	내 번호가 1이고 지금이 2이면 옛 응답입니다

4. 흐름

- **글자를 입력할 때:** 타이머를 취소하고, 검색어가 지금과 같으면 끝냅니다. 다르면 0.3초 타이머를 겁니다.
- **정렬을 바꾸거나 Enter를 누를 때:** 타이머를 취소하고 바로 `refreshList({ query, sort, page: 1 })`를 부릅니다.
- **`refreshList`:** 목록을 받고, 실패했거나 옛 응답이면 `null`로 끝냅니다. 아니면 목록과 페이지 줄을 그립니다.

5. 왜 이렇게

선택	이유
0.3초 디바운스	템플릿과 HTML 주석에 적힌 값입니다. 칠 때마다 요청하면 서버에 부담이 되고 목록이 깜빡입니다

선택	이유
정렬과 Enter는 바로 요청	한 번 고르는 동작이라 기다릴 이유가 없습니다. Enter는 "지금 찾아"라는 뜻입니다
검색어가 같으면 건너뜀	공백만 더 쳤거나, 지웠다가 다시 친 경우에 같은 결과를 또 받지 않습니다
검색과 정렬은 항상 1쪽부터	3쪽을 보다가 검색하면 결과가 1쪽뿐일 수 있습니다
옛 응답 버리기 (30번에 추가)	`beta` 요청의 응답이 늦게 도착하면, 그 뒤에 보낸 `alpine`의 결과를 덮어써서 화면과 주소가 `beta`로 돌아갑니다. 그래서 요청마다 번호를 붙이고, 마지막 번호의 응답만 씁니다. **시험에서 이 두 줄을 잠시 빼고 돌려 보니 실제로 `Beta Beat`가 남는 것을 확인**했고, 다시 되돌렸습니다
`refreshList`로 묶음	"받기 → 그리기 → 페이지 줄"이 32, 33, 35, 37번에서 반복되기 때문입니다
`type="search"` 칸의 × 버튼	누르면 `input` 이벤트가 일어나서 칸을 비운 것과 같게 동작합니다 (따로 처리하지 않습니다)

6. 시험 (실제 myProject.html, 실제 서버, 6개 + 앞의 시험 다시)

프로젝트 4개(`Alpha Song`, `Beta Beat`, `Gamma Loop`, `alpine`)를 만들고, 글자 입력과 이벤트를 페이지 안에서 일으켰습니다. `window.fetch`를 감싸서 목록 요청 수를 셌습니다.

#	한 일	결과
01	`alp`를 0.06초 간격으로 입력	치는 동안 요청은 0번이고, 멈춘 뒤 **1번** 가며, `alpine, Alpha Song`, `?query=alp`가 됩니다
02	공백 하나를 더 입력	요청 0번
03	정렬을 Name으로	바로 1번 가고, `Alpha Song, alpine`, `?query=alp&sort=title`이 됩니다
04	검색 칸 비움	1번 가고, 4개, `?sort=title`이 됩니다
05	`gam` 입력 후 바로 Enter	0.15초 안에 `Gamma Loop`가 나오고, 끝까지 요청은 **1번**입니다 (타이머가 취소되었습니다)
06	`beta` 요청이 1.2초 늦게 도착하게 하고, 그사이 `alpine` + Enter	화면과 주소가 `alpine` 그대로입니다 (옛 응답을 버렸습니다). 가드를 빼면 `Beta Beat`로 덮어씌워집니다

앞의 시험 다시 (30번과 32번을 고쳤으므로): 30번의 14개와 32번의 10개가 모두 그대로 통과했습니다. 페이지 넘김 클릭은 여전히 요청 1번이고 `?page=2`가 됩니다.

JS 34 — `initRowMenus()`

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면: 줄의 ··· 메뉴 (PRD 01 FR-124, FR-125)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	33 `initSearch()` · 35 `canOpenStudio()`

1. 하는 일

목록 줄 오른쪽의 `···` 메뉴에서 누른 **Edit details나 Delete가 어느 프로젝트에 대한 것인지** 기억하고, 모달 칸을 채우고, 메뉴를 닫습니다. 모달을 여는 일은 HTML 버튼의 `command="show-modal"` 속성이 이미 합니다.

사용자가 한 일	결과
`···` → Edit details	`selected`가 그 프로젝트가 되고, Edit 모달의 제목 칸과 설명 칸에 지금 값이 들어가며, 지난 오류 문구가 지워지고, 메뉴가 닫힙니다
설명이 없는 줄의 Edit details	설명 칸은 **빈 칸**입니다 (화면 글자 `No description`이 들어가지 않습니다)
`···` → Delete	`selected`가 그 프로젝트가 되고, 메뉴가 닫힙니다 (확인창은 41번 `initDeleteProject`가 띄웁니다)
메뉴를 연 채로 다른 줄의 `···`	앞의 메뉴는 닫히고, 새 메뉴만 열립니다
메뉴 바깥을 누름	열린 메뉴가 닫힙니다
메뉴 안에서 Esc	메뉴가 닫히고, 포커스는 `···` 버튼으로 돌아갑니다

31번 `buildRow`에 두 줄 추가: 줄에 `data-title`과 `data-description`을 넣습니다. 화면의 제목 글자에는 `Not opened`가, 설명 글자에는 `No description`이 섞여 있어서, 그대로 읽으면 수정 칸에 잘못된 값이 들어가기 때문입니다.

2. 언제 쓰나

호출하는 곳	화면
페이지 시작 (`myproject.js` 맨 아래, 이미 있음)	`initRowMenus();`로 한 번만 부릅니다
`selected`를 읽는 곳	36번 `initEditProjectForm`(저장할 프로젝트 번호), 37번 `initDeleteProject`(지울 번호, 확인창 제목)

이벤트 위임(event delegation)은 줄마다 버튼에 이벤트 처리를 거는 대신 **목록(`ul`) 하나에만** 걸고, 눌린 곳에서 `closest('[data-action]')`으로 버튼을 찾는 방법입니다. 검색이나 페이지 넘김으로 줄이 다시 그려져도 `ul`은 그대로이므로 다시 걸 필요가 없습니다.

3. 코드

import (`myproject.js` 맨 위: 30번과 같고, 이번에 추가한 것은 없습니다)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio } from '../core/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

이 함수는 import를 쓰지 않습니다.

상수 (이번에 추가)

```
/** ··· 메뉴에서 고른 프로젝트 — Edit details · Delete 가 어느 프로젝트인지 (34 가 채움, 36 · 37 이 읽음)
*/
export const selected = { id: null, title: '', description: '' };
```

함수

```
export function initRowMenus() {
  const list = document.getElementById('project-list');
  if (!list) return;

  // ··· 메뉴 안 버튼 — 목록에 하나만 걸어 둠 (줄을 다시 그려도 동작)
  list.addEventListener('click', function (event) {
    const button = event.target.closest('[data-action]');
    if (!button) return;
    const row = button.closest('li[data-project-id]');
    selected.id = Number(row.dataset.projectId);
    selected.title = row.dataset.title;
    selected.description = row.dataset.description;

    if (button.dataset.action === 'edit') {
      const title = document.getElementById('edit-title');
      const description = document.getElementById('edit-description');
      const error = document.getElementById('edit-project-error');
      if (title) title.value = selected.title;
      if (description) description.value = selected.description;
      if (error) { error.hidden = true; error.textContent = ''; }   // 지난번 실패 문구 지우기
    }
    const menu = button.closest('details');
    if (menu) menu.open = false;
  });

  // 메뉴 바깥을 누르면 열린 메뉴 닫기 (다른 줄의 ··· 를 눌러도 앞 메뉴는 닫힘)
  document.addEventListener('click', function (event) {
    const menus = document.querySelectorAll('.pj-menu[open]');
    for (let i = 0; i < menus.length; i++) {
      if (!menus[i].contains(event.target)) menus[i].open = false;
    }
  });

  // Esc → 열린 메뉴 닫고, 메뉴 안에 있던 포커스는 ··· 버튼으로 돌려줌
  document.addEventListener('keydown', function (event) {
    if (event.key !== 'Escape') return;
    const menus = document.querySelectorAll('.pj-menu[open]');
```

```
    for (let i = 0; i < menus.length; i++) {
      const hadFocus = menus[i].contains(document.activeElement);
      menus[i].open = false;
      if (hadFocus) menus[i].querySelector('summary').focus();
    }
  });
}
```

31번 `buildRow`에 추가한 줄

```
  row.dataset.title = item.title;                                    // 34 가 수정 칸을 채울 때 씀
  row.dataset.description = item.description ? item.description : '';
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
for (const menu of document.querySelectorAll('.pj-menu[open]')) {
  if (!menu.contains(event.target)) menu.open = false;
}
row.dataset.description = item.description ?? '';
error?.setAttribute('hidden', '');
```

최신 문법	실제 코드	뜻
`for (const x of 목록)`	`for (let i = 0; i < 목록.length; i++)`	하나씩 돌면서 처리합니다
`a ?? ''`	`a ? a : ''`	없으면 빈 문자열을 씁니다
`error?.…`	`if (error) …`	있을 때만 처리합니다

문법 풀이

import 문법	뜻
(이 함수는 import 없음)	

식	뜻	예
`event.target.closest('[data-action]')`	누른 곳에서 위로 올라가며 `data-action`이 있는 요소를 찾습니다	버튼 안의 글자를 눌러도 버튼을 찾습니다
`row.dataset.projectId`	`data-project-id` 속성 값(문자열)이며, `Number(…)`로 숫자로 바꿉니다	`'1801'` → 1801
`details.open = false`	`<details>` 메뉴를 닫습니다 (`open` 속성 제거)	
`document.querySelectorAll('.pj-menu[open]')`	열려 있는 메뉴들만 고릅니다	
`menus[i].contains(event.target)`	누른 곳이 그 메뉴 안인지 확인합니다	바깥이면 닫습니다
`document.activeElement`	지금 포커스가 있는 요소입니다	Esc를 누르기 전의 `Edit details` 버튼

4. 흐름

1. **목록을 클릭하면:** 버튼을 찾고, 그 줄의 번호, 제목, 설명을 `selected`에 넣습니다. Edit이면 수정 칸을 채우고 오류를 지우며, Delete면 삭제 모달의 제목을 정합니다. 그리고 메뉴를 닫습니다.
2. **문서를 클릭하면:** 열린 메뉴 중에서 누른 곳을 품지 않은 메뉴를 닫습니다.

3. **Esc를 누르면:** 열린 메뉴를 닫고, 포커스가 메뉴 안에 있었다면 `···`로 옮깁니다.

5. 왜 이렇게

선택	이유
목록 하나에만 이벤트를 검 (위임)	줄은 검색하거나 페이지를 넘길 때마다 다시 그려집니다. 줄마다 걸면 다시 그릴 때마다 다시 걸어야 합니다 (템플릿)
`data-title`과 `data-description`에서 읽음	화면 글자에서 읽으면 `Neon Rain Not opened`나 `No description`이 수정 칸에 들어갑니다
`selected`를 밖으로 내보냄	저장(36)과 삭제(37)는 다른 함수이므로, 어느 프로젝트인지 넘겨받을 곳이 필요합니다
지난 오류 문구를 지움	앞의 프로젝트를 저장하다 실패한 문구가 다른 프로젝트의 모달에 남지 않게 합니다
바깥 클릭과 Esc로 닫음	`<details>`는 다시 눌러야만 닫힙니다. 일반 메뉴처럼 동작하게 했습니다 (템플릿)
Esc 뒤에 포커스를 `···`로 옮김	키보드 사용자가 메뉴 안의 버튼에 있다가 메뉴가 닫히면 포커스가 사라지기 때문입니다
(41에서 바꿈) Delete 모달 제목 채우기를 지움	41번이 HTML 삭제 모달을 `confirmDialog`로 바꾸면서 모달이 없어졌습니다. 처음에는 여기서 `#delete-project-name`을 채웠습니다

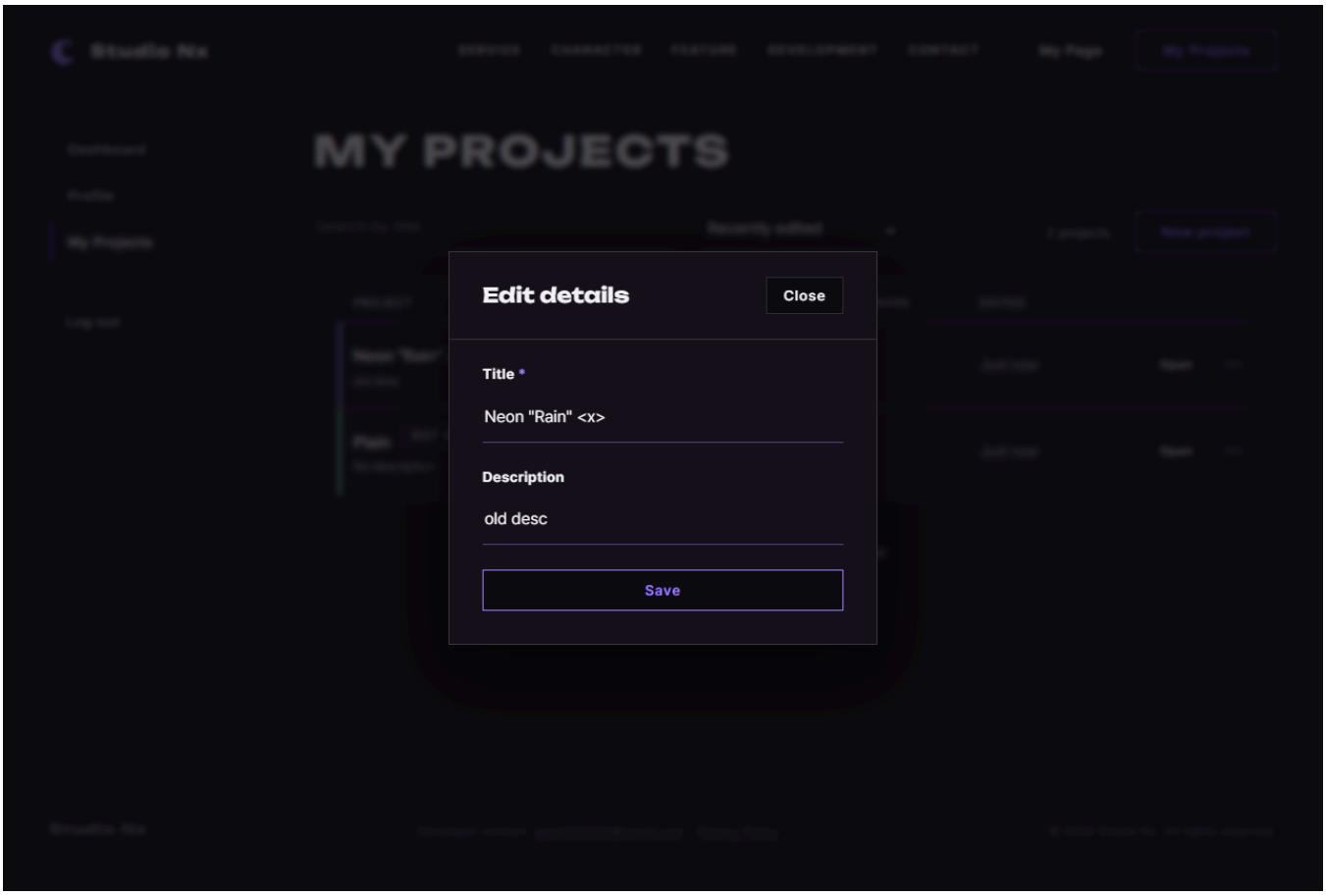
참고: 모달은 버튼의 `command` 속성으로 열립니다 (최신 Chrome과 Edge가 지원합니다). 지원하지 않는 브라우저를 위한 `initModalFallback`(`core/modal.js`)은 아직 템플릿 상태이며, 이 작업의 범위 밖입니다.

6. 시험 (실제 myProject.html, 실제 서버, 7개)

프로젝트 `Plain`과 `Neon "Rain" <x>`(설명 `old desc`)를 만들고, 페이지 안에서 버튼을 눌렀습니다.

#	한 일	결과
01	첫 줄의 `···` → Edit details	메뉴가 열렸다가 닫히고, Edit 모달이 열리며, 제목은 `Neon "Rain" <x>`, 설명은 `old desc`이고, `selected.id`가 맞습니다
02	(오류 문구를 띄워 둔 채로) 설명이 없는 줄의 Edit	설명 칸은 빈 칸이고, 오류는 숨겨지며 글자도 지워집니다
03	첫 줄의 Delete	(41 이후 다시 시험) 확인창 `Delete project?`가 뜨고, 제목 글자는 그대로이며, `selected`가 맞고, 메뉴가 닫힙니다
04	첫 메뉴를 연 채로 둘째 줄의 `···`	첫째는 닫히고 둘째가 열립니다
05	페이지 제목을 누름	둘 다 닫힙니다
06	메뉴 안의 `Edit details`에 포커스를 두고 Esc	메뉴가 닫히고, 포커스가 `···`로 갑니다
07	이름순으로 다시 그린 뒤 Edit	그대로 동작합니다

캡처: Edit 모달에 채워진 값



JS 35 — `canOpenStudio()` (core/studio.js)

항목	내용
파일	`studionx/frontend/static/js/core/studio.js`
작업	P1-14 내 프로젝트 화면: 시퀀서를 열 수 있는 화면인지 판단 (PRD 01 FR-129, PRD 00 반응형 기준)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	34 `initRowMenus()` · 36 `showDesktopOnly()`

1. 하는 일

지금 브라우저의 화면 폭이 **1280px 이상**이면 `true`를, 아니면 `false`를 돌려줍니다. 시퀀서(`studioNx.html`)는 넓은 화면에서만 열기로 했기 때문입니다 (PRD 00, 01 FR-129).

화면 폭	결과	예
1920, 1440, 1280	`true`	노트북, 데스크톱
1279, 1024	`false`	작은 노트북 창, 태블릿 가로
390	`false`	휴대폰

순서를 바꾼 이유: 새 프로젝트 기능(원래 35번)은 이 파일의 `canOpenStudio`, `openStudio`, `showDesktopOnly`를 쓰는데, `core/studio.js`의 네 함수가 모두 아직 템플릿 상태였습니다. 이 파일을 처음 쓰는 페이지가 myProject이므로 여기서 먼저 하나씩 구현합니다. 순서는 **35 `canOpenStudio` → 36 `showDesktopOnly` → 37 `openStudio` → 38 `initDesktopOnlyLinks` → 39 `initNewProjectForm` → 40 수정 → 41 삭제 → 42 페이지 시작**입니다. (myPage의 `initDesktopOnlyLinks`도 P1-17 때 그대로 씁니다.)

2. 언제 쓰나

호출하는 곳	화면
39 `initNewProjectForm`	프로젝트를 만든 뒤, 넓은 화면이면 시퀀서 창을 열고, 좁은 화면이면 "컴퓨터에서 여세요" 안내를 띄웁니다
38 `initDesktopOnlyLinks`	목록의 Open이나 제목을 누를 때 씁니다 (myProject, myPage)

`matchMedia`는 CSS의 `@media (min-width: 1280px)` 조건을 JS에서 그대로 검사하는 브라우저 기능입니다. `window.innerWidth >= 1280`과 거의 같지만, 세로 스크롤바 폭 처리까지 **CSS와 똑같이** 판단합니다. 그래서 화면 모양(CSS)과 JS의 판단이 1px 차이로 어긋나지 않습니다.

3. 코드

import (`studio.js` 맨 위)

```
export const STUDIO_MIN_WIDTH = 1280;
```

`studio.js`는 다른 파일을 import하지 않습니다. 이 함수는 같은 파일의 상수 `STUDIO_MIN_WIDTH`를 씁니다.

함수

```
export function canOpenStudio() {
  return window.matchMedia('(min-width: ' + STUDIO_MIN_WIDTH + 'px)').matches;
}
```

최신 문법으로 쓰면 (공부용)

```
export const canOpenStudio = () => matchMedia(`(min-width: ${STUDIO_MIN_WIDTH}px)`).matches;
```

최신 문법	실제 코드	뜻
`` `(min-width: ${STUDIO_MIN_WIDTH}px)` ``	`'(min-width: ' + STUDIO_MIN_WIDTH + 'px)'`	문자열 안에 값을 넣습니다
`const f = () => 값`	`function f() { return 값; }`	한 줄 함수입니다

문법 풀이

import 문법	뜻
(import 없음)	`window.matchMedia`는 브라우저에 원래 있습니다

식	뜻	예
`window.matchMedia('(min-width: 1280px)')`	그 조건의 검사기를 만듭니다	
`.matches`	지금 조건에 맞는지(true / false)입니다. **부를 때마다 새로** 검사합니다	창을 줄인 뒤 부르면 false입니다

4. 흐름

1. `'(min-width: 1280px)'` 조건을 만듭니다.
2. 지금 조건에 맞는지를 돌려줍니다.

5. 왜 이렇게

선택	이유
`matchMedia`를 씀 (템플릿)	CSS 반응형 기준과 같은 방식이라서, 스크롤바와 확대 처리가 CSS와 같습니다
부를 때마다 검사 (값을 저장해 두지 않음)	사용자가 창 크기를 바꿀 수 있습니다. 버튼을 누른 그 순간의 폭이 기준이 되어야 합니다
CSS 픽셀 기준	고해상도 화면(DPR 2, 3)도 CSS 폭으로 판단합니다. 1280 CSS px이면 실제 픽셀이 2560이어도 같은 레이아웃입니다. 브라우저를 확대(Ctrl +)하면 CSS 폭이 줄어서 false가 될 수 있는데, 그만큼 화면이 실제로 좁아진 것이므로 그대로 따릅니다
1280은 상수 `STUDIO_MIN_WIDTH`로 둠	다른 곳(안내 문구, 시험)에서도 같은 숫자를 씁니다

6. 시험 (실제 myProject.html, 헤드리스 Chrome, 7개)

한 페이지를 열어 둔 채 **화면 폭만 바꿔 가며** 불렀습니다 (새로 열지 않았으므로, 부를 때마다 다시 검사한다는 것을 확인했습니다).

화면 폭 (DPR)	결과
1920 (1)	`true`
1280 (1)	`true`: 경계값을 포함합니다
1279 (1)	`false`
1024 (1)	`false`
390 (3, 휴대폰)	`false`
1280 (2, 고해상도)	`true`
1440 (1)	`true`: 다시 넓히면 다시 true가 됩니다

돌려준 값은 모두 `boolean`이었습니다.

JS 36 — `showDesktopOnly()` (core/studio.js)

항목	내용
파일	`studionx/frontend/static/js/core/studio.js`
작업	P1-14 내 프로젝트 화면: 작은 화면 안내 모달 (PRD 01 FR-129)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	35 `canOpenStudio()` · 37 `openStudio()`

1. 하는 일

화면이 좁아서 시퀀서를 열 수 없을 때(35번이 false일 때) **"컴퓨터에서 여세요" 모달**(`#desktop-only-modal`, `static/partials/desktop-only-modal.html`)을 띄웁니다. 상황에 맞는 문장 하나만 보이게 하고, 제목을 채웁니다.

부른 값	보이는 문장
`showDesktopOnly('created', 'Summer Loop')`	**Summer Loop** is saved. 작은 화면에서 프로젝트를 **만든 직후**에 씁니다 (39)
`showDesktopOnly('open', 'Summer Loop')`	**Summer Loop** opens in the studio, which needs a larger screen. Open이나 제목을 **눌렀을 때** 씁니다 (38)
그 밖의 값 (`'xyz'`)	`open` 문장을 보여 줍니다

두 경우 모두 아래에 "Sign in on a computer and it'll be waiting at the top of your dashboard."와 Got it 버튼이 나옵니다.

2. 언제 쓰나

호출하는 곳	화면
39 `initNewProjectForm`	작은 화면에서 만들기에 성공하면 `'created'`로 부릅니다
38 `initDesktopOnlyLinks`	작은 화면에서 Open이나 제목을 누르면 `'open'`으로 부릅니다 (myProject, myPage)

`<dialog>`와 `showModal()`: HTML의 기본 모달입니다. `showModal()`로 열면 뒤의 화면이 막히고, Esc로 닫힙니다. 이미 열린 모달에 다시 `showModal()`을 부르면 오류(`InvalidStateError`)가 나므로, `modal.open`을 먼저 확인합니다.

3. 코드

import (`studio.js` 맨 위)

```
export const STUDIO_MIN_WIDTH = 1280;
```

`studio.js`에는 import가 없습니다. 이 함수는 브라우저의 기본 기능만 씁니다.

함수

```
export function showDesktopOnly(variant, title) {
  const modal = document.getElementById('desktop-only-modal');
```

```
  if (!modal) return;
  if (variant !== 'created') variant = 'open';

  // 상황에 맞는 문단만 보이기
  const leads = modal.querySelectorAll('[data-variant]');
  for (let i = 0; i < leads.length; i++) {
    leads[i].hidden = leads[i].dataset.variant !== variant;
  }
  const name = document.getElementById('desktop-only-' + variant + '-name');
  if (name) name.textContent = title;

  if (!modal.open) modal.showModal();
}
```

최신 문법으로 쓰면 (공부용)

```
for (const lead of modal.querySelectorAll('[data-variant]')) lead.hidden = lead.dataset.variant !== vari
ant;
document.getElementById(`desktop-only-${variant}-name`)?.replaceChildren(title);
modal.open || modal.showModal();
```

최신 문법	실제 코드	뜻
`for (const x of 목록)`	`for (let i = 0; …; i++)`	하나씩 처리합니다
`el?.replaceChildren(title)`	`if (name) name.textContent = title`	있을 때만 글자를 넣습니다
`a || b()`	`if (!a) b()`	a가 참이 아니면 b를 실행합니다

문법 풀이

import 문법	뜻
(import 없음)	

식	뜻	예
`modal.querySelectorAll('[data-variant]')`	모달 안에서 `data-variant`가 있는 문단들을 찾습니다	created와 open 두 개
`leads[i].hidden = 조건`	조건이 true면 숨깁니다	고른 것과 다르면 숨깁니다
`'desktop-only-' + variant + '-name'`	고른 문장 안에 있는 제목 칸의 id입니다	`desktop-only-created-name`
`name.textContent = title`	글자로만 넣습니다	`Night <b>Drive</b>`가 그 글자 그대로 나옵니다
`modal.open`	지금 열려 있는지 확인합니다	

4. 흐름

1. 모달이 없으면 끝냅니다.
2. 모르는 값이면 `'open'`으로 바꿉니다.
3. 고른 문장만 보이게 합니다.
4. 제목을 넣습니다.
5. 모달이 닫혀 있으면 엽니다.

5. 왜 이렇게

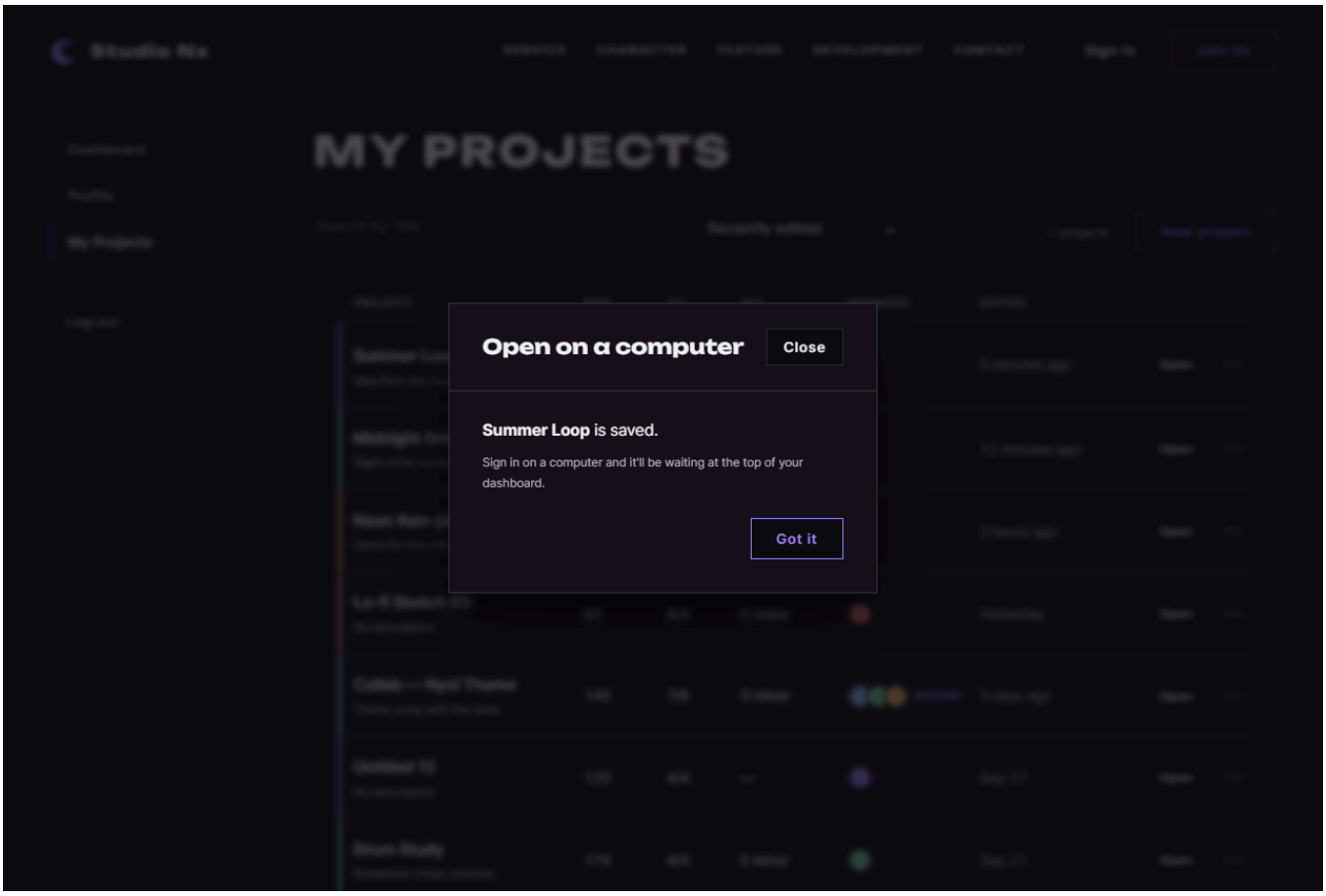
선택	이유
문장 두 개를 HTML에 두고 숨기기만 함	글자는 HTML(화면 담당)이 정하고 JS는 고르기만 합니다. 문구를 고칠 때 JS를 건드리지 않아도 됩니다 (템플릿)
`textContent`	제목은 사용자가 입력한 글자이므로 태그로 해석되지 않게 합니다 (31번과 같습니다)
모르는 값은 `'open'`으로 처리	부르는 쪽이 잘못된 값을 넘겨도 모달이 빈 문장으로 뜨지 않게 합니다
이미 열려 있으면 다시 열지 않음	`showModal()`을 두 번 부르면 오류가 납니다. 문장과 제목만 바뀝니다
모달이 없으면 조용히 끝냄	공통 조각(`includePartials`)이 아직 끼워지지 않은 페이지에서 불려도 오류가 나지 않게 합니다

6. 시험 (실제 myProject.html, 헤드리스 Chrome, 5개)

보이는지는 계산된 CSS로 확인했습니다.

#	한 일	결과
01	`('created', 'Summer Loop')`	모달이 열리고, created 문장만 보이며, 제목은 `Summer Loop`입니다
02	닫은 뒤 `('open', 'Night <b>Drive</b>')`	open 문장만 보이고, 제목은 글자 그대로입니다 (`<b>` 요소 0개)
03	열린 채로 `('created', 'Second')`	오류가 나지 않고, 문장이 created로 바뀌며, 제목은 `Second`입니다
04	`('xyz', 'Fallback')`	open 문장이 나오고, 제목은 `Fallback`입니다
05	Got it을 누름	모달이 닫힙니다

캡처: 만든 직후의 안내 (`created`)



JS 37 — `openStudio()` (+ `openBlankStudio`) (core/studio.js)

항목	내용
파일	`studionx/frontend/static/js/core/studio.js`
작업	P1-14 내 프로젝트 화면: 시퀀서를 별도 창으로 열기 (PRD 01 FR-126, PRD 02 FR-258, 2026-10-05 결정)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	36 `showDesktopOnly()` · 38 `initDesktopOnlyLinks()`

1. 하는 일

시퀀서(`studioNx.html?projectId=번호`)를 **별도의 브라우저 창**으로 엽니다. 프로젝트 목록 창을 닫아도 작업 창은 남게 하려는 결정(2026-10-05)에 따른 것입니다.

부른 값	결과
`openStudio(5)`	화면 크기의 창 `studio-5`에 `studioNx.html?projectId=5`를 엽니다
같은 프로젝트로 또 `openStudio(5)`	**새 창을 열지 않고**, 이미 열린 `studio-5` 창을 앞으로 가져옵니다
`openStudio(7, { isNew: true })`	`…?projectId=7&new=1`이 되어 로딩 화면이 "새 프로젝트" 모드로 나옵니다
`openStudio(9, { win: 빈창 })`	미리 열어 둔 빈 창의 주소를 `…?projectId=9`로, 이름을 `studio-9`로 바꿉니다
팝업이 막힘 (`window.open`이 null)	**지금 탭**에서 시퀀서로 이동합니다

함께 만든 것: `openBlankStudio()`. 새 프로젝트(39)는 서버가 번호를 돌려줘야 주소를 알 수 있습니다. 그런데 브라우저는 **클릭한 바로 그 순간**(사용자 동작 안)에만 새 창을 허락하므로, 응답을 기다린 뒤에 열면 팝업으로 막힙니다. 그래서 클릭할 때 빈 창을 먼저 열고(`openBlankStudio`), 응답이 오면 `openStudio(id, { win })`로 주소를 넣습니다.

2. 언제 쓰나

호출하는 곳	화면
38 `initDesktopOnlyLinks`	넓은 화면에서 Open이나 제목을 누르면 `openStudio(id)`를 부릅니다 (myProject, myPage)
39 `initNewProjectForm`	Create를 누르면 `openBlankStudio()`를 부르고, 응답 뒤에 `openStudio(id, { isNew: true, win })`를 부릅니다. 실패하면 빈 창을 닫습니다

팝업 차단과 사용자 동작(user activation): 브라우저는 클릭이나 키 입력 직후의 짧은 동안에만 `window.open`을 허락합니다. 한 번 허락받아 창을 열면 그 허락은 **이미 쓴 것**이 되므로, 같은 클릭으로 창을 또 열면 막힙니다 (시험 05에서 확인).

창 이름: `window.open(주소, 이름)`에서 같은 이름의 창이 이미 있으면 새로 만들지 않고 그 창을 씁니다. 프로젝트마다 `studio-번호`라는 이름을 붙여서, 같은 프로젝트가 창 두 개로 열리지 않게 합니다 (협업과 저장 충돌 방지).

3. 코드

import (`studio.js` 맨 위)

```
export const STUDIO_MIN_WIDTH = 1280;
```

`studio.js`에는 import가 없습니다. 이 함수는 같은 파일의 `STUDIO_WINDOW_PREFIX('studio-')`를 씁니다.

함수 (`openBlankStudio`와 창 옵션 `studioWindowFeatures` 포함)

```
export function openStudio(projectId, options = {}) {
  let url = 'studioNx.html?projectId=' + encodeURIComponent(projectId);
  if (options.isNew) url += '&new=1';
  const name = STUDIO_WINDOW_PREFIX + projectId;

  // 미리 열어 둔 빈 창이 있으면 그 창에서 (새 프로젝트 – 응답을 기다린 뒤라 새로 열면 팝업으로 막힘)
  let win = options.win;
  if (win && !win.closed) {
    win.name = name;            // 다음에 같은 프로젝트를 누르면 이 창이 앞으로 오게
    win.location.href = url;
  } else {
    win = window.open(url, name, studioWindowFeatures());
  }

  if (!win) {
    location.href = url;        // 팝업이 막힘 → 같은 탭에서 열기
    return null;
  }
  win.focus();
  return win;
}

/**
 * 시퀀서용 빈 창을 먼저 열기 – 클릭 처리 안에서 **await 전에** 부를 것 (39 새 프로젝트)
 * 응답이 오면 openStudio(id, { win }) 로 주소를 넣고, 실패하면 win.close()
 * (구현: AI – 2026-10-08, 37 과 함께)
 * @returns {Window|null}  팝업이 막혔으면 null
 */
export function openBlankStudio() {
  return window.open('', '', studioWindowFeatures());
}

/** 창 옵션 – 'popup' 이 있어야 탭이 아니라 별도 창, 크기는 화면에 맞춤 */
function studioWindowFeatures() {
  return 'popup,width=' + screen.availWidth + ',height=' + screen.availHeight + ',left=0,top=0';
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
export function openStudio(projectId, { isNew = false, win = null } = {}) {
  const url = `studioNx.html?projectId=${encodeURIComponent(projectId)}${isNew ? '&new=1' : ''}`;
  …
  win?.focus();
}
```

```
const studioWindowFeatures = () => `popup,width=${screen.availWidth},height=${screen.availHeight},left=0,top=0`;
```

최신 문법	실제 코드	뜻
`{ isNew = false, win = null } = {}` (구조 분해 매개변수)	`options = {}` + `options.isNew`, `options.win`	객체에서 칸을 꺼냅니다
템플릿 문자열	`'…' + 값 + '…'`	문자열을 잇습니다
`win?.focus()`	`if (win) win.focus()`	있을 때만 실행합니다

문법 풀이

import 문법	뜻
(import 없음)	`window.open`과 `screen`은 브라우저에 원래 있습니다

식	뜻	예
`encodeURIComponent(projectId)`	주소에 넣어도 안전한 글자로 바꿉니다	숫자는 그대로입니다
`window.open(url, name, features)`	새 창(또는 같은 이름의 창)을 열고 그 창 객체를 돌려줍니다. 막히면 `null`입니다	
`'popup,width=…,height=…,left=0,top=0'`	`popup`이 있으면 탭이 아니라 **별도의 창**으로 엽니다	
`screen.availWidth`, `availHeight`	작업 표시줄을 뺀 화면 크기입니다	1920 × 1040
`win.closed`	사용자가 그 창을 닫았는지 확인합니다	
`win.name = name`	빈 창에 나중에 이름을 붙입니다 (같은 사이트의 창이라서 가능합니다)	`''` → `'studio-9'`
`win.location.href = url`	그 창의 주소를 바꿉니다	
`location.href = url`	**지금 탭**의 주소를 바꿉니다	팝업이 막혔을 때

4. 흐름

1. 주소(`studioNx.html?projectId=…`, 새 프로젝트면 `&new=1`을 더함)와 창 이름(`studio-…`)을 만듭니다.
2. 미리 열어 둔 빈 창이 있고 아직 열려 있으면 그 창에 이름과 주소를 넣습니다. 없으면 `window.open`으로 엽니다.
3. 창을 열지 못했으면(막힘) 지금 탭에서 열고 `null`을 돌려줍니다. 열었으면 그 창을 앞으로 가져오고 돌려줍니다.

5. 왜 이렇게

선택	이유
별도 창 (`popup`)	목록 창을 닫아도 작업 창이 남게 합니다 (2026-10-05 결정, PRD 02 FR-258). 새 탭이면 같은 창에 속하므로 함께 닫힙니다
창 이름 `studio-번호`	같은 프로젝트를 다시 누르면 창이 하나 더 생기지 않고, 그 창이 앞으로 옵니다 (시험 02)
빈 창을 먼저 엶 (`openBlankStudio`)	새 프로젝트는 응답을 기다려야 번호를 알 수 있습니다. 기다린 뒤에 열면 팝업이 차단됩니다 (템플릿 설명 그대로)

선택	이유
빈 창에 나중에 이름을 붙임	열 때는 번호를 몰라서 이름이 없습니다. 주소를 넣을 때 `studio-번호`로 붙여서, 다음 클릭에서 같은 창을 찾을 수 있게 합니다
사용자가 빈 창을 닫았으면 새로 시도	닫힌 창에는 주소를 넣을 수 없습니다. 다만 클릭 허락은 빈 창이 이미 썼으므로 보통 막히고, 그러면 아래 줄의 같은 탭 열기로 이어집니다 (시험 05)
막히면 같은 탭에서 엶	팝업을 막아 둔 사용자도 시퀀서에는 들어갈 수 있게 합니다 (템플릿)
창 크기 = 화면 크기	시퀀서는 넓을수록 좋고, 1280 이상의 화면에서만 열립니다 (35)

6. 시험 (실제 myProject.html, 헤드리스 Chrome, 6개)

CDP로 "사용자가 누른 것"처럼 실행했고(`userGesture`), 열린 창은 Chrome의 창 목록으로도 확인했습니다.

#	한 일	결과
01	`openStudio(5)`	`window.open('studioNx.html?projectId=5', 'studio-5', 'popup,width=800,height=600,left=0,top=0')` (시험 화면 800×600)으로 창이 열리고, 이름은 `studio-5`이며, 창 목록에 추가됩니다
02	다시 `openStudio(5)`	**같은 창 객체**를 돌려주고, 창 수는 그대로입니다
03	`openStudio(7, { isNew: true })`	`?projectId=7&new=1`, `studio-7`
04	`openBlankStudio()` → 0.5초 기다림 (응답 흉내) → `openStudio(9, { isNew: true, win })`	빈 창(`about:blank`, 이름 없음)이 **같은 창 그대로** `?projectId=9&new=1`, `studio-9`가 됩니다
05	빈 창을 열고 바로 닫은 뒤 `openStudio(11, { win })`	클릭 허락을 빈 창이 이미 썼으므로 새 창은 막히고, **지금 탭**이 `studioNx.html?projectId=11`로 이동합니다 (막힘 대비가 동작했습니다)
06	`window.open`이 null을 돌려주게 하고 `openStudio(3)`	`null`을 돌려주고, 지금 탭이 `studioNx.html?projectId=3`으로 이동합니다

처음에는 05에서 "새 창이 열린다"고 예상했지만, 실제로는 허락이 이미 쓰여서 막히고 같은 탭으로 열렸습니다. 코드는 그대로 두고(막히면 같은 탭에서 여는 것이 의도한 대비입니다), 설명을 실제 동작에 맞췄습니다.

JS 38 — `initDesktopOnlyLinks()` (core/studio.js)

항목	내용
파일	`studionx/frontend/static/js/core/studio.js`
작업	P1-14 내 프로젝트 화면: 시퀀서 링크 클릭 처리 (PRD 01 FR-126, FR-129)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	37 `openStudio()` · 39 `initNewProjectForm()`

1. 하는 일

목록의 **제목이나 Open**처럼 `studioNx.html`로 가는 링크를 누르면, 지금 탭에서 넘어가는 대신 화면 폭에 따라 다르게 처리합니다.

사용자 동작	결과
넓은 화면(1280 이상)에서 **그냥 클릭**	시퀀서가 **새 브라우저 창**으로 열립니다 (37). 같은 프로젝트라면 이미 열린 창이 앞으로 옵니다
좁은 화면에서 그냥 클릭	"Open on a computer" 안내 모달이 뜹니다 (36). 제목에는 `Not opened` 표시가 붙지 않습니다
Ctrl, ⌘, Shift, Alt + 클릭, 휠 클릭	**브라우저 기본 동작 그대로**입니다 (새 탭, 새 창, 내려받기)
시퀀서가 아닌 링크	그대로 둡니다

사용자 질문 (2026-10-08): "Ctrl 클릭을 해야 해?" 아닙니다. **그냥 클릭 한 번으로 새 창이 열립니다.** Ctrl+클릭 등은 "새 탭으로 열기"에 익숙한 사람의 습관을 **막지 않는다**는 뜻입니다. 창과 탭 중에서 사용자가 고르는 설정은 Phase 2의 환경 설정(PRD 02 FR-2164)에 있습니다.

2. 언제 쓰나

호출하는 곳	범위
`myproject.js` 맨 아래 (이미 있음)	`initDesktopOnlyLinks(document.getElementById('project-list'))`로 목록에 겁니다
`mypage.js` (P1-17, 이미 있음)	`initDesktopOnlyLinks(document.getElementById('recent-list'))`로 최근 프로젝트에 겁니다

링크의 `href`는 그대로 둡니다. JS가 꺼져 있거나 오류가 나도 링크는 지금 탭에서 열립니다 (점진적 향상).

3. 코드

import (`studio.js` 맨 위)

```
export const STUDIO_MIN_WIDTH = 1280;
```

`studio.js`에는 import가 없습니다. 이 함수는 같은 파일의 `canOpenStudio`(35), `showDesktopOnly`(36), `openStudio`(37)를 씁니다.

함수 (제목 찾기는 `studioLinkTitle`로 나눴습니다)

```
export function initDesktopOnlyLinks(root = document) {
  if (!root) return;
  root.addEventListener('click', function (event) {
    // 새 탭 · 새 창 · 내려받기 같은 브라우저 기본 동작은 막지 않음
    if (event.defaultPrevented || event.button !== 0) return;
    if (event.ctrlKey || event.metaKey || event.shiftKey || event.altKey) return;
    const link = event.target.closest('a[href^="studioNx.html"]');
    if (!link) return;

    event.preventDefault();
    if (!canOpenStudio()) {
      showDesktopOnly('open', studioLinkTitle(link));
      return;
    }
    const params = new URLSearchParams(link.getAttribute('href').split('?')[1]);
    openStudio(params.get('projectId'));
  });
}

/** 링크가 속한 프로젝트의 제목 — 줄의 data-title, 없으면 <b> 안 첫 글자 마디 */
function studioLinkTitle(link) {
  const row = link.closest('[data-title]');
  if (row) return row.dataset.title;
  const holder = link.querySelector('b') ? link : link.closest('li, section, article, div');
  const bold = holder ? holder.querySelector('b') : null;
  return bold && bold.firstChild ? bold.firstChild.textContent.trim() : 'This project';
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
if (event.ctrlKey || event.metaKey || event.shiftKey || event.altKey) return;   // 같음
const projectId = new URL(link.href).searchParams.get('projectId');
return link.closest('[data-title]')?.dataset.title
    ?? holder?.querySelector('b')?.firstChild?.textContent.trim()
    ?? 'This project';
```

최신 문법	실제 코드	뜻
`a?.b` 사슬	`a ? a.b : null`, `if`	있을 때만 다음으로 넘어갑니다
`x ?? y`	`x ? x : y`	없으면 다음 값을 씁니다
`new URL(link.href).searchParams`	`new URLSearchParams(href.split('?')[1])`	주소의 `?` 뒤 값을 읽습니다

문법 풀이

import 문법	뜻
(import 없음)	

식	뜻	예
`event.button !== 0`	왼쪽 버튼이 아니라는 뜻입니다 (1 = 휠, 2 = 오른쪽)	휠 클릭이면 브라우저가 새 탭을 엽니다

식	뜻	예
`event.ctrlKey`, `metaKey`, `shiftKey`, `altKey`	누른 채로 클릭한 키입니다 (meta = Mac의 ⌘)	Ctrl = 새 탭, Shift = 새 창, Alt = 내려받기
`event.defaultPrevented`	앞에서 다른 코드가 이미 막았는지 확인합니다	
`'a[href^="studioNx.html"]'`	`href`가 `studioNx.html`로 **시작하는** 링크입니다	
`link.getAttribute('href').split('?')[1]`	`?` 뒤의 부분입니다	`'projectId=7'`
`params.get('projectId')`	그 값입니다	`'7'`
`link.closest('[data-title]')`	위로 올라가며 `data-title`이 있는 요소를 찾습니다 (myProject의 줄, 31)	
`bold.firstChild.textContent`	`<b>`의 첫 글자 마디만 읽습니다. 뒤의 `<span>Not opened</span>`는 빠집니다	`Night Drive`

4. 흐름

1. 범위(`root`) 안에서 클릭이 일어나면, 이미 막힌 클릭이거나 왼쪽 버튼이 아니거나 보조 키를 누른 경우에는 그대로 둡니다.
2. 시퀀서 링크가 아니면 그대로 두고, 맞으면 기본 이동을 막습니다.
3. 좁은 화면이면 제목을 찾아서 안내 모달을 띄우고, 넓은 화면이면 주소의 `projectId`로 새 창을 엽니다.

5. 왜 이렇게

선택	이유
그냥 클릭하면 새 창	목록 창을 닫아도 작업 창이 남게 합니다 (2026-10-05 결정)
보조 키와 휠 클릭은 그대로 둠	사용자가 **직접 고른** 동작(새 탭 등)을 JS가 바꾸면 불편합니다. 템플릿의 Ctrl, ⌘, 휠에 Shift와 Alt도 더했습니다
범위(`root`) 하나에만 이벤트를 검	목록이 다시 그려져도 동작합니다 (34번과 같은 위임 방식)
제목은 `data-title`을 먼저 읽음	화면 글자에는 `Not opened`가 섞여 있습니다 (31). myPage처럼 `data-title`이 없으면 `<b>`의 첫 마디를 읽습니다
제목을 찾지 못하면 `This project`	모달이 빈 이름으로 뜨지 않게 합니다
`href`를 남겨 둠	JS 없이도 링크가 동작하고, 마우스를 올리면 주소가 보입니다

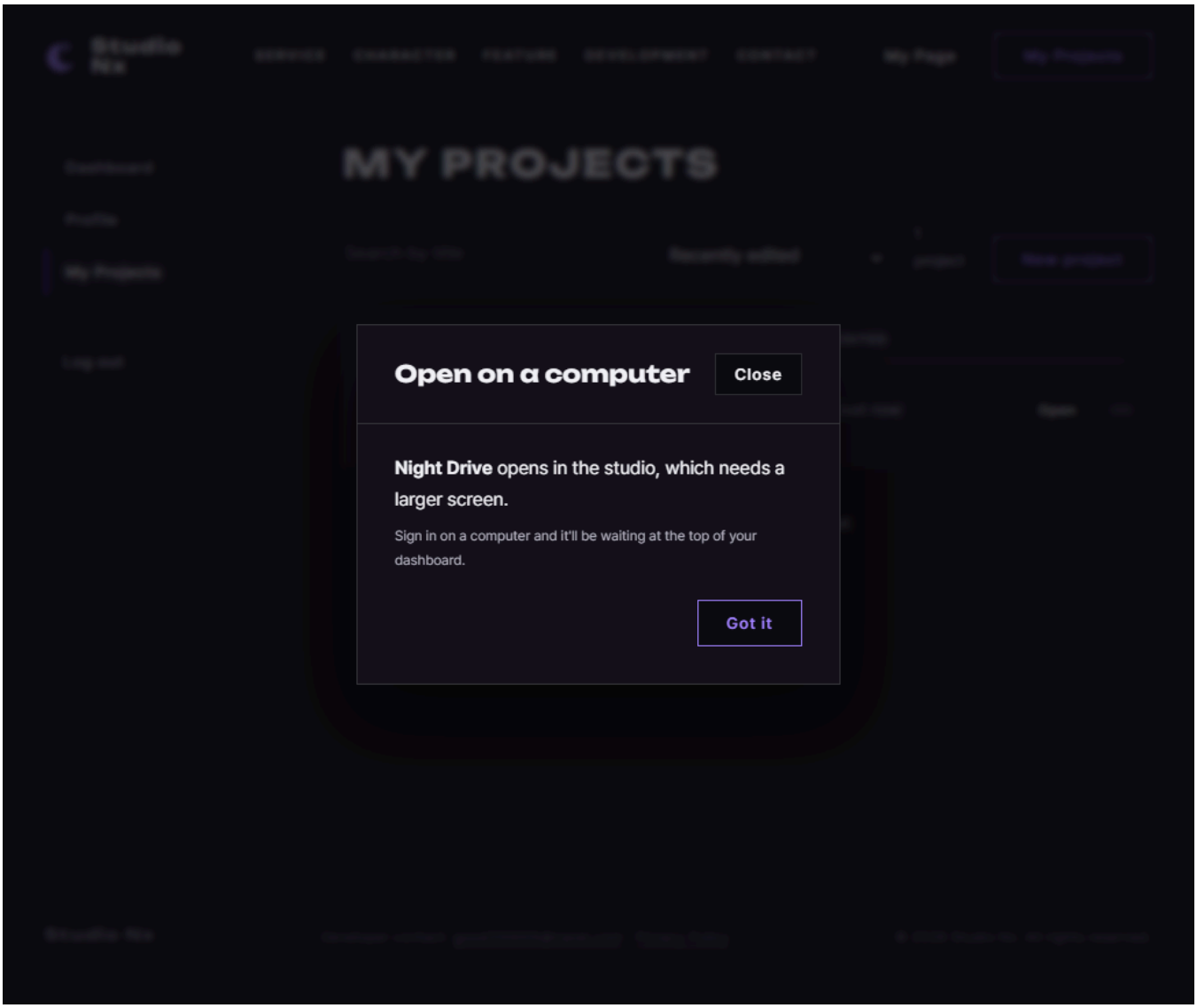
P1-17 참고: myPage의 'Ready to start' 배너는 제목이 `<h2>`라서 지금 규칙으로는 `This project`가 됩니다. 배너는 `recent-list` 밖에 있어서 지금은 이 함수의 범위가 아니며, P1-17에서 배너에 `data-title`을 넣으면 그대로 맞게 됩니다.

6. 시험 (실제 myProject.html, 실제 서버, 헤드리스 Chrome, 6개)

프로젝트 `Night Drive`를 만들어 목록을 그리고, "사용자가 누른 것"처럼 클릭했습니다. 클릭이 막혔는지는 문서 끝에서 `defaultPrevented`로 기록했습니다.

#	한 일	결과
01	1280 화면에서 제목을 그냥 클릭	기본 이동을 막고, **새 창** `studioNx.html?projectId=…`이 생기며, 지금 탭은 myProject 그대로입니다
02	같은 줄의 Open 버튼	기본 이동을 막고, 창 수는 그대로입니다 (같은 창이 앞으로 옵니다)
03	Ctrl, Shift, Alt, Meta, 휠 클릭	5번 모두 **막지 않고**, 모달도 뜨지 않습니다
04	1024 화면에서 제목 클릭	기본 이동을 막고, 안내 모달이 뜨며, 제목은 `Night Drive`입니다 (`Not opened` 없음). 새 창은 생기지 않습니다
05	`data-title`이 없는 myPage 모양의 줄	안내 모달의 제목이 `Midnight Drive`입니다
06	목록 안의 다른 링크 (`userInfo.html`)	막지 않고, 모달도 뜨지 않습니다

캡처: 좁은 화면(1024)에서 제목을 눌렀을 때



JS 39 — `initNewProjectForm()`

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면: 새 프로젝트 만들기 (PRD 01 FR-122, FR-126, FR-129)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	38 `initDesktopOnlyLinks()` · 40 `initEditProjectForm()`

1. 하는 일

New project 모달에서 **Create project**를 누르면 서버에 프로젝트를 만들고(`POST /projects`, P1-13 59), 화면 폭에 따라 다음 단계로 이어 갑니다.

상황	결과
넓은 화면 (1280 이상)	누른 순간 **빈 새 창**이 열리고, 응답이 오면 그 창이 `studioNx.html?projectId=…&new=1`(시퀀서의 "새 프로젝트" 로딩)로 바뀝니다
좁은 화면	창을 열지 않고 **"… is saved."** 안내를 띄웁니다 (36 `created`)
두 경우 모두	모달을 닫고 폼을 비운 뒤, 검색과 정렬을 기본으로 되돌리고 목록을 다시 받습니다. 그래서 **새 프로젝트가 맨 위**에 나옵니다
제목이 공백뿐	보내지 않고 `Enter a title.`을 보여 줍니다
BPM 10(20~999 밖) 등	브라우저 검사에서 멈추고 알림창을 띄우며(FIX-01 `initFormAlerts`), 보내지 않습니다
서버 오류	미리 열어 둔 빈 창을 **닫고**, 모달 안에 서버 문구를 보여 줍니다. 입력값은 그대로 둡니다
보내는 동안	Create 버튼을 잠급니다 (두 번 눌러서 프로젝트가 두 개 만들어지지 않게)

보내는 값은 `{ title, description(없으면 null), bpm(숫자), time_sig_num, time_sig_den }`입니다. 박자 칸의 `'7/8'`은 7과 8로 나눠서 보냅니다.

2. 언제 쓰나

호출하는 곳	화면
페이지 시작 (`myproject.js` 맨 아래, 이미 있음)	`initNewProjectForm();`으로 폼에 한 번 걸어 둡니다

모달은 `New project` 버튼의 `command="show-modal"`이 엽니다 (HTML). 이 함수는 **보내기**만 맡습니다.

3. 코드

import (`myproject.js` 맨 위)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
```

```
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio, openBlankStudio } from '../co
re/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

`openBlankStudio`를 이번에 추가했습니다. 이 함수는 `api`(P1-02), `canOpenStudio`(35), `showDesktopOnly`(36), `openStudio`와 `openBlankStudio`(37), 같은 파일의 `refreshList`(33)를 씁니다.

함수

```
export function initNewProjectForm() {
  const form = document.getElementById('new-project-form');
  if (!form) return;
  const modal = document.getElementById('new-project-modal');
  const error = document.getElementById('new-project-error');
  const submit = form.querySelector('[type="submit"]');
  const showError = function (message) {
    error.textContent = message;
    error.hidden = false;
  };

  form.addEventListener('submit', async function (event) {
    event.preventDefault();
    const title = document.getElementById('new-title').value.trim();
    const description = document.getElementById('new-description').value.trim();
    const sig = document.getElementById('new-sig').value.split('/');   // '7/8' → ['7', '8']
    error.hidden = true;
    if (!title) {
      showError('Enter a title.');
      return;
    }

    // 넓은 화면: 클릭한 지금 빈 창을 열어 둠 (응답 뒤에 열면 팝업으로 막힘 — 37)
    const wide = canOpenStudio();
    const win = wide ? openBlankStudio() : null;
    submit.disabled = true;
    let project;
    try {
      project = await api.post('/projects', {
        title: title,
        description: description ? description : null,
        bpm: Number(document.getElementById('new-bpm').value),
        time_sig_num: Number(sig[0]),
        time_sig_den: Number(sig[1]),
      });
    } catch (err) {
      if (win) win.close();
      showError(err.message);
      return;
    } finally {
      submit.disabled = false;
    }

    // 성공 — 모달 닫고 폼 비우기
    if (modal && modal.open) modal.close();
    form.reset();
```

```
    if (wide) {
      openStudio(project.project_id, { isNew: true, win: win });
    } else {
      showDesktopOnly('created', project.title);
    }

    // 목록은 기본 조건(최근 수정순 · 검색 없음)으로 — 새 프로젝트가 맨 위
    const query = document.getElementById('project-query');
    const sort = document.getElementById('project-sort');
    if (query) query.value = '';
    if (sort) sort.value = 'updated';
    refreshList({ query: '', sort: 'updated', page: 1 });
  });
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
const [num, den] = document.getElementById('new-sig').value.split('/').map(Number);
project = await api.post('/projects', { title, description: description || null, bpm: +bpmField.value, t
ime_sig_num: num, time_sig_den: den });
win?.close();
openStudio(project.project_id, { isNew: true, win });
```

최신 문법	실제 코드	뜻
`const [a, b] = 배열`	`sig[0]`, `sig[1]`	배열에서 값을 꺼냅니다
`.map(Number)`	`Number(sig[0])`를 따로 씀	모두 숫자로 바꿉니다
`{ title }`, `{ win }`	`{ title: title }`, `{ win: win }`	줄임 속성입니다
`x || null`	`x ? x : null`	비었으면 null입니다
`win?.close()`	`if (win) win.close()`	있을 때만 닫습니다

문법 풀이

import 문법	뜻
`import { …, openBlankStudio } from '../core/studio.js'`	클릭할 때 빈 시퀀서 창을 미리 여는 함수입니다 (37)

식	뜻	예
`form.addEventListener('submit', async function (event) { … })`	Enter나 Create를 누르면 실행됩니다 (브라우저 검사를 통과한 뒤에만 불립니다)	
`'7/8'.split('/')`	`/`로 나눕니다	`['7', '8']`
`Number('128.5')`	문자열을 숫자로 바꿉니다	128.5
`try { … } catch { … } finally { … }`	실패하면 catch를, **성공이든 실패든** 끝에 finally를 실행합니다	버튼 잠금 풀기
`catch` 안의 `return` 뒤에도 `finally`가 실행됨		
`form.reset()`	칸을 HTML의 처음 값으로 되돌립니다 (제목 빈 칸, BPM 120, 4/4)	
`modal.close()`	`<dialog>`를 닫습니다	

4. 흐름

1. 기본 보내기(페이지 이동)를 막습니다. 제목이 공백뿐이면 문구를 보여 주고 끝냅니다.
2. 넓은 화면이면 **바로** 빈 창을 엽니다 (응답을 기다리기 전에).
3. 버튼을 잠그고 `POST /projects`를 보냅니다.
4. 실패하면 빈 창을 닫고, 문구를 보여 주고, 버튼을 풉니다.
5. 성공하면 모달을 닫고 폼을 비웁니다. 넓은 화면이면 빈 창을 시켜서로 바꾸고, 좁은 화면이면 "saved" 안내를 띄웁니다. 그리고 검색 칸과 정렬 칸을 기본으로 되돌린 뒤 목록을 다시 받습니다(33 `refreshList`).

5. 왜 이렇게

선택	이유
빈 창을 **await 전에** 엶	응답 뒤에 열면 팝업이 차단됩니다 (37)
실패하면 빈 창을 닫음	빈 흰 창이 남지 않게 합니다 (시험 05에서 창 수가 그대로임을 확인했습니다)
공백뿐인 제목은 보내지 않음	`required`는 공백 한 칸도 통과시킵니다. 서버 422의 영어 검사 문구보다 짧고 분명한 문구를 보여 주려는 것입니다
검사는 브라우저에 맡김 (BPM 범위, 필수 칸)	HTML에 `min`, `max`, `required`가 이미 있고, FIX-01이 말풍선 대신 알림창으로 보여 줍니다
만든 뒤에는 검색과 정렬을 기본으로 되돌림	검색 중(`zzz`)이었다면 새 프로젝트가 목록에 보이지 않습니다. "맨 위에 보인다"는 템플릿의 요구를 지키기 위해서입니다. 검색 칸과 정렬 칸의 글자도 함께 비웁니다
좁은 화면에서는 "saved" 안내를 띄움	시켜서를 열 수 없는 화면이므로, 프로젝트가 만들어졌다는 것과 컴퓨터에서 열면 된다는 것을 알려 줍니다 (FR-129)
목록 창은 그대로 둠	시켜서는 새 창으로 열리고, 이 창은 갱신된 목록으로 남습니다 (템플릿)

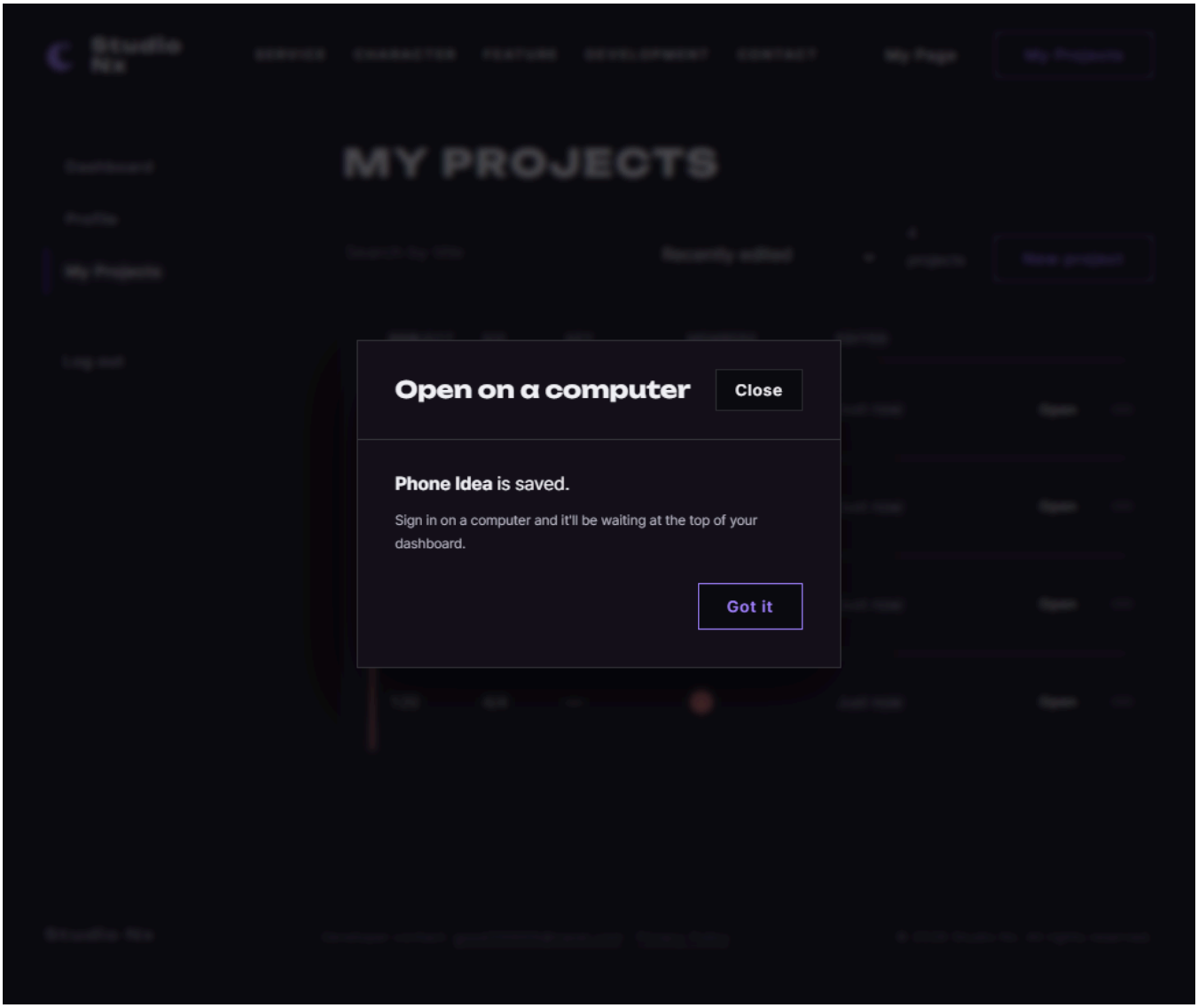
6. 시험 (실제 myProject.html, 실제 서버, 헤드리스 Chrome, 7개)

New project 버튼으로 모달을 열고 칸을 채운 뒤 보냈습니다 ("사용자가 누른 것"처럼). 열린 창은 Chrome의 창 목록으로 확인했습니다.

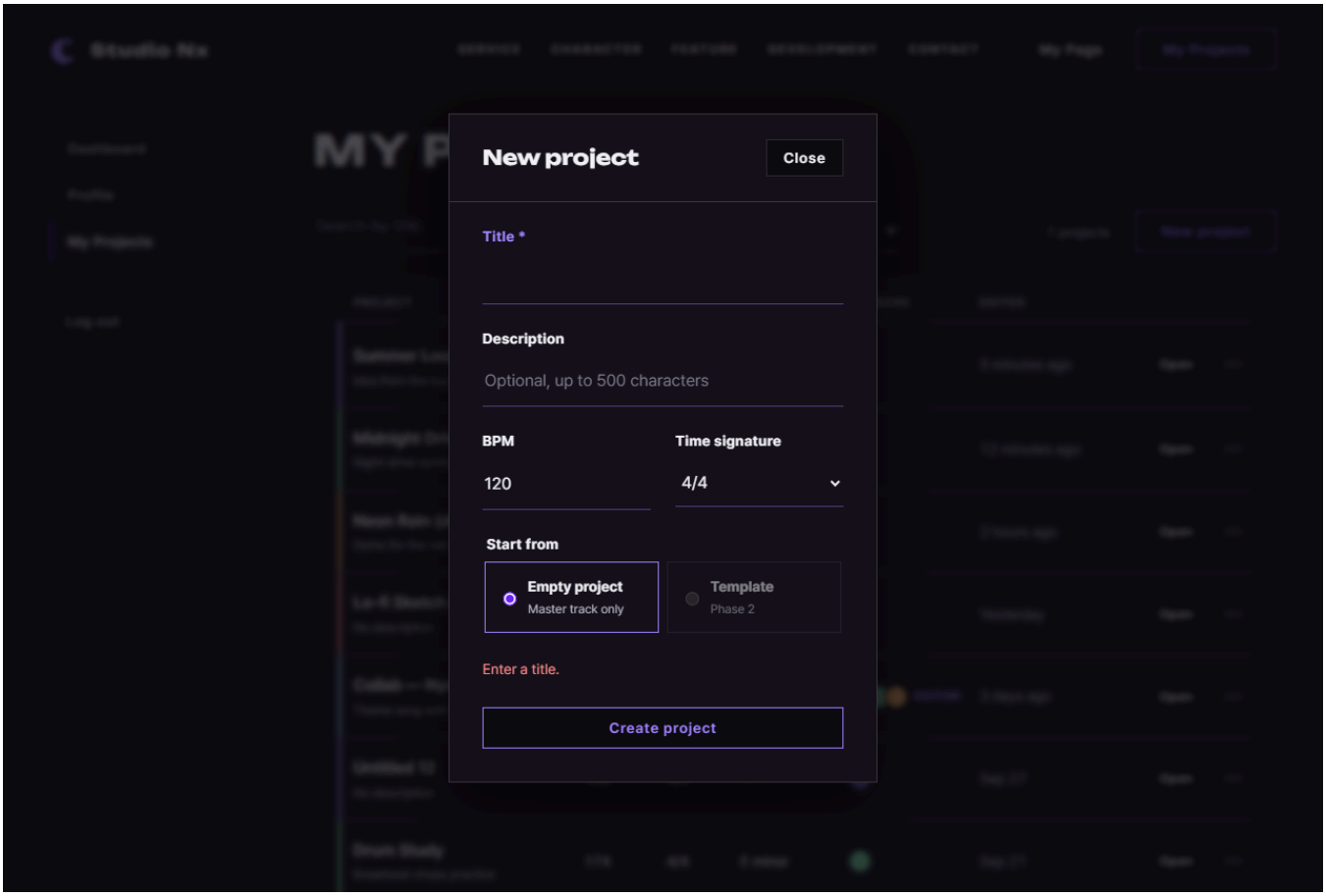
#	한 일	결과
01	1280, `' New Wave '`, 설명, 128.5, 7/8	새 창 `studioNx.html?projectId=…&new=1`이 열리고, 모달이 닫히며, 제목 칸이 비워지고, 목록 맨 위에 `New Wave`가 나옵니다. 서버에 저장된 값은 제목 `New Wave`, 설명, bpm `128.50`, `7/8`입니다
02	검색 `zzz`(0개) 중에 `Second` 만들기	목록이 기본 조건으로 돌아가서 `Second, New Wave, Old One`이 나오고, 주소의 `?query=zzz`가 없어지며, 검색 칸이 비워집니다
03	1024, `Phone Idea`, 3/4	새 창은 열리지 않고, "Phone Idea is saved." 안내가 뜨며, 목록 맨 위에 나옵니다
04	제목이 공백	`Enter a title.`이 나오고, 요청은 0번이며, 창도 열리지 않고, 모달은 그대로입니다
05	서버 500 (가짜)	문구 `Something went wrong. Please try again.`이 나오고, 빈 창이 닫혀서 창 수는 그대로이며, 버튼이 풀리고, 입력값은 그대로입니다
06	응답을 0.8초 늦춤	보내는 동안 버튼이 잠기고, 끝나면 풀리며, 요청은 1번입니다
07	BPM 10	요청은 0번이고, 알림창(FIX-01)이 뜹니다

캡처

좁은 화면에서 만든 뒤:


MY PROJECTS
Open on a computer
Close
Phone Idea is saved.
Sign in on a computer and it'll be waiting at the top of your dashboard.
Got it

공백 제목:


New project
Close
Title *
Description
Optional, up to 500 characters
BPM
120
Time signature
4/4
Start from
Empty project
Master track only
Template
Phase 2
Enter a title.
Create project

JS 40 — `initEditProjectForm()`

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면: 제목과 설명 수정 (PRD 01 FR-124)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	39 `initNewProjectForm()` · 41 `initDeleteProject()`

1. 하는 일

Edit details 모달의 **Save**를 누르면 `PATCH /projects/{id}`(P1-13 60)로 제목과 설명을 저장하고, 성공하면 **그 줄만** 새 값으로 다시 그립니다.

상황	결과
제목 `' Beta v2 '`, 설명 `new`	모달이 닫히고, 그 줄이 `Beta v2 · new`로 바뀝니다 (자리는 그대로). 다른 줄은 그대로입니다
설명을 비우거나 공백만 넣음	설명을 지우므로 줄에는 `No description`이 나오고, 서버 값은 `null`이 됩니다
제목이 공백	보내지 않고 `Enter a title.`을 보여 줍니다
그사이 다른 창에서 지워진 프로젝트	`Project not found.`를 보여 주고, 목록을 다시 받아서 그 줄이 사라지게 합니다
VIEWER (31번에서 Edit이 잠겨 있지만, 혹시 보내면)	서버 문구 `You don't have permission.`을 보여 줍니다
서버 오류	모달 안에 문구를 보여 주고, 입력값은 그대로 두며, 버튼을 풉니다

2. 언제 쓰나

호출하는 곳	화면
페이지 시작 (`myproject.js` 맨 아래, 이미 있음)	`initEditProjectForm();`
함께 쓰는 것	34번 `initRowMenus`가 `selected`와 칸 값을 채우고 모달을 엽니다. 이 함수는 저장만 맡습니다

3. 코드

import (`myproject.js` 맨 위: 39번과 같고, 이번에 추가한 것은 없습니다)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio, openBlankStudio } from '../co
re/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

이 함수는 `api`(P1-02)와 같은 파일의 `selected`(34), `current`(30), `buildRow`(31), `refreshList`(33)를 씁니다.

함수

```
export function initEditProjectForm() {
  const form = document.getElementById('edit-project-form');
  if (!form) return;
  const modal = document.getElementById('edit-project-modal');
  const error = document.getElementById('edit-project-error');
  const submit = form.querySelector('[type="submit"]');
  const showError = function (message) {
    error.textContent = message;
    error.hidden = false;
  };

  form.addEventListener('submit', async function (event) {
    event.preventDefault();
    if (selected.id === null) return;
    const title = document.getElementById('edit-title').value.trim();
    const description = document.getElementById('edit-description').value.trim();
    error.hidden = true;
    if (!title) {
      showError('Enter a title.');
      return;
    }

    submit.disabled = true;
    let item;
    try {
      item = await api.patch('/projects/' + selected.id, {
        title: title,
        description: description ? description : null,   // 비우면 설명 지우기
      });
    } catch (err) {
      showError(err.message);
      if (err.status === 404) refreshList({ query: current.query, sort: current.sort, page: current.page
});
      return;
    } finally {
      submit.disabled = false;
    }

    if (modal && modal.open) modal.close();
    selected.title = item.title;
    selected.description = item.description ? item.description : '';
    const row = document.querySelector('#project-list > li[data-project-id="' + item.project_id + '"]');
    const template = document.getElementById('project-row-template');
    if (row && template) row.replaceWith(buildRow(template, item));
  });
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
item = await api.patch(`/projects/${selected.id}`, { title, description: description || null });
selected.description = item.description ?? '';
```

```
document.querySelector(`#project-list > li[data-project-id="${item.project_id}"]`)?.replaceWith(buildRow(template, item));
```

최신 문법	실제 코드	뜻
템플릿 문자열	`'/projects/' + selected.id`	문자열을 잇습니다
`a ?? ''`	`a ? a : ''`	없으면 빈 문자열입니다
`el?.replaceWith(…)`	`if (row && template) row.replaceWith(…)`	있을 때만 실행합니다

문법 풀이

import 문법	뜻
(새 import 없음)	`api.patch(주소, 본문)`은 PATCH 요청을 보내며, 토큰은 자동으로 붙습니다 (P1-02)

식	뜻	예
`description ? description : null`	빈 문자열이면 null을 보내서 서버가 설명을 지우게 합니다 (P1-13 56에서 `""`도 None으로 처리하지만, 뜻을 분명히 하려고 씁니다)	
`'li[data-project-id="' + id + '"]'`	그 번호의 줄을 찾습니다	
`row.replaceWith(새 요소)`	그 자리에 새 줄을 끼우고 옛 줄을 뺍니다	순서와 다른 줄은 그대로입니다
`err.status === 404`	프로젝트가 없다는 뜻입니다 (지워졌거나 권한을 잃음, P1-13 57)	

4. 흐름

1. 기본 보내기를 막습니다. 고른 프로젝트가 없으면 끝내고, 제목이 공백이면 문구를 보여 줍니다.
2. 버튼을 잠그고 `PATCH /projects/{id} { title, description }`을 보냅니다.
3. 실패하면 문구를 보여 주고, 404이면 목록을 다시 받으며, 버튼을 풉니다.
4. 성공하면 모달을 닫고, `selected`를 새 값으로 바꾼 뒤, 그 줄을 `buildRow`로 다시 만들어 바꿔 끼웁니다.

5. 왜 이렇게

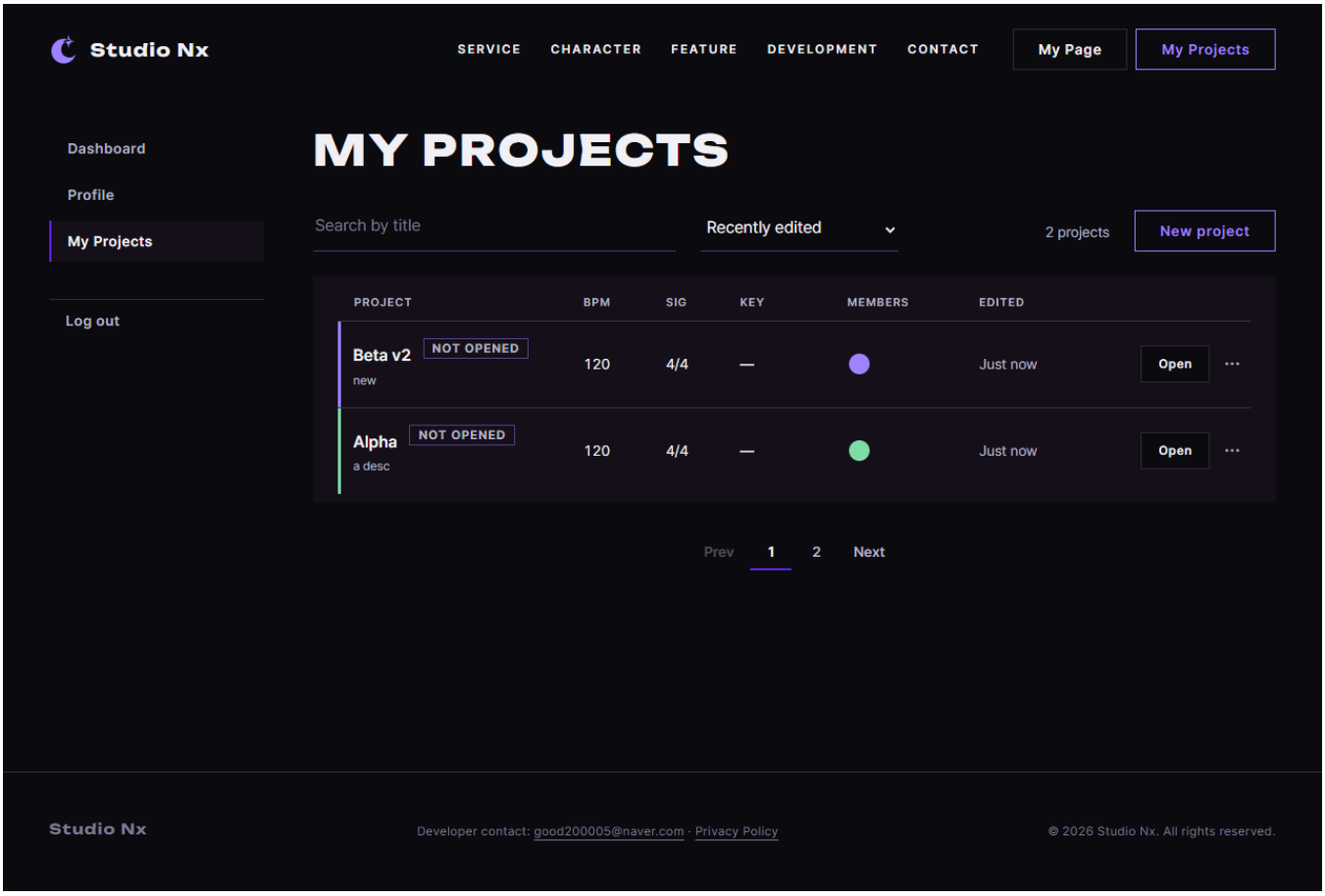
선택	이유
그 줄만 다시 그림	템플릿대로입니다. 응답(`ProjectItem`)에 줄을 그리는 데 필요한 값이 모두 있으므로 목록 전체를 다시 받지 않아도 됩니다 (P1-13 62)
줄의 자리를 옮기지 않음	최근 수정순이라면 원래 맨 위로 가야 하지만, 고친 줄이 눈앞에서 사라지면 사용자가 결과를 확인하기 어렵습니다. 다음에 새로고침하거나 검색할 때 위로 올라갑니다
`buildRow`를 다시 씀	`Not opened`, 역할, 잠금 등 줄의 규칙이 31번과 똑같이 적용됩니다. `data-title`도 새 값으로 바뀌므로, 다음 Edit에서 새 값이 채워집니다 (시험 02)
공백뿐인 제목은 보내지 않음	39번과 같은 이유입니다
404이면 목록을 다시 받음	지워진 프로젝트의 줄이 남아 있으면 사용자가 또 누르게 됩니다
설명을 비우면 `null`로 보냄	수정 칸을 비우는 것은 설명을 지운다는 뜻입니다 (P1-13 60의 표와 같습니다)

6. 시험 (실제 myProject.html, 실제 서버, 5개)

프로젝트 `Alpha`(설명 `a desc`)와 `Beta`를 만들고, ⋯ → Edit details로 모달을 연 뒤 저장했습니다.

#	한 일	결과
01	Beta를 `' Beta v2 '`, `new`로 수정	모달이 닫히고, 줄이 `Beta v2 · new`가 되며(첫 줄 그대로), `Alpha` 줄은 그대로입니다. 서버에는 `Beta v2`, `new`가 저장되고, PATCH는 1번입니다
02	다시 Edit을 열고 설명을 공백으로	모달에 새 값(`Beta v2`, `new`)이 채워져 있고, 저장한 뒤에는 `No description`이 나오며, 서버 값은 `null`입니다
03	Alpha의 제목을 공백으로	`Enter a title.`이 나오고, PATCH는 0번이며, 모달은 그대로입니다
04	Edit을 연 뒤 다른 곳에서 Alpha를 지우고 저장	`Project not found.`가 나오고, 목록을 다시 받아서 Alpha 줄이 사라집니다
05	서버 500 (가짜)	문구가 나오고, 모달은 그대로이며, 버튼이 풀립니다

캡처: 저장한 뒤 (`Beta v2 · new`)



JS 41 — `initDeleteProject()`

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`, `studionx/frontend/myProject.html`
작업	P1-14 내 프로젝트 화면: 프로젝트 삭제 (PRD 01 FR-125, FIX-01 확인창, 2026-10-07 결정)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	40 `initEditProjectForm()` · 42 `loadInitialList()`

1. 하는 일

`···` → **Delete**를 누르면 확인창(`confirmDialog`, FIX-01 20)을 띄우고, **Delete project**를 고르면 `DELETE /projects/{id}`(P1-13 61, 소프트 삭제)를 보낸 뒤 목록을 다시 받습니다.

상황	결과
Delete → 확인창	제목 `Delete project?`, 문장 `“P03” will be removed from your projects. Collaborators will lose access too.`, 빨간 `Delete project` 버튼이 나옵니다 (처음 커서는 Cancel에 있습니다)
Cancel, Esc, 바깥 클릭	아무 일도 일어나지 않습니다
Delete project	삭제한 뒤 같은 검색, 정렬, 페이지로 다시 받습니다 (`12 projects` → `11 projects`)
2쪽의 마지막 한 줄을 지움	빈 2쪽을 그리지 않고 **1쪽으로** 갑니다
OWNER가 아님	Delete 버튼이 잠겨 있으므로(31) 확인창도 뜨지 않습니다
서버 오류	알림창 `Couldn’t delete`와 서버 문구를 보여 주고, 버튼을 다시 켭니다
그사이 다른 곳에서 지워짐 (404)	알림창 `Project not found.`를 보여 주고 목록을 다시 받습니다

HTML도 바꿨습니다 (2026-10-07 결정대로): `#delete-project-modal` `<dialog>`를 지우고, 목록 줄과 틀(`<template>`)의 Delete 버튼 7곳에서 `command="show-modal" commandfor="delete-project-modal"`을 뺐습니다. **34번 `initRowMenus`에서도** 삭제 모달의 제목을 채우던 세 줄을 지웠습니다 (모달이 없어졌기 때문입니다).

2. 언제 쓰나

호출하는 곳	화면
페이지 시작 (`myproject.js` 맨 아래, 이미 있음)	`initDeleteProject();`로 목록에 한 번 겁니다

확인창(`confirmDialog`)은 브라우저 기본 `confirm()` 대신 쓰는 SweetAlert2 창입니다 (FIX-01 20). `await confirmDialog({...})`는 확인이면 `true`, 취소나 Esc나 바깥 클릭이면 `false`를 돌려줍니다. `danger: true`이면 확인 버튼이 빨강이고 처음 커서가 **Cancel**에 있으므로, Enter를 바로 눌러도 지워지지 않습니다.

3. 코드

import (`myproject.js` 맨 위)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio, openBlankStudio } from '../co
re/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts, confirmDialog, showAlert } from '../core/dialog.js';
```

`confirmDialog`와 `showAlert`를 이번에 추가했습니다. 이 함수는 `api`, `confirmDialog`, `showAlert`와 같은 파일의 `current`, `loadProjects`, `renderProjects`, `renderPager`, `refreshList`를 씁니다.

함수

```
export function initDeleteProject() {
  const list = document.getElementById('project-list');
  if (!list) return;

  list.addEventListener('click', async function (event) {
    const button = event.target.closest('[data-action="delete"]');
    if (!button || button.disabled) return;
    const row = button.closest('li[data-project-id]');
    const id = row.dataset.projectId;
    const title = row.dataset.title;

    const ok = await confirmDialog({
      title: 'Delete project?',
      text: '“' + title + '” will be removed from your projects. Collaborators will lose access too.',
      confirmText: 'Delete project',
      danger: true,
    });
    if (!ok) return;

    button.disabled = true;   // 두 번 누름 방지 (성공하면 줄이 다시 그려짐)
    try {
      await api.delete('/projects/' + id);
    } catch (err) {
      button.disabled = false;
      await showAlert('Couldn’t delete', err.message, { tone: 'error' });
      if (err.status === 404) refreshList({ query: current.query, sort: current.sort, page: current.page
});
      return;
    }

    // 같은 조건으로 다시 받기 — 마지막 줄을 지워 그 페이지가 비면 (빈 화면을 그리지 않고) 한 페이지 앞으로
    const page = current.page;
    const data = await loadProjects({ query: current.query, sort: current.sort, page: page });
    if (!data) return;
    if (data.items.length === 0 && page > 1) {
      refreshList({ query: current.query, sort: current.sort, page: page - 1 });
      return;
    }
    renderProjects(data);
    renderPager(data.total, data.page, data.size);
```

```
  });
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
const { projectId: id, title } = row.dataset;
text: `“${title}” will be removed from your projects. Collaborators will lose access too.`,
const { query, sort, page } = current;
```

최신 문법	실제 코드	뜻
`const { projectId: id, title } = row.dataset`	`const id = row.dataset.projectId` 등 두 줄	칸을 꺼내면서 이름을 바꿉니다
템플릿 문자열	`'“' + title + '” will …'`	문자열을 잇습니다
`const { query, sort, page } = current`	`current.query` 등을 그대로 씀	칸을 꺼냅니다

문법 풀이

import 문법	뜻
`import { …, confirmDialog, showAlert } from '../core/dialog.js'`	확인창(true/false)과 알림창입니다 (FIX-01 20, 18)

식	뜻	예
`event.target.closest('[data-action="delete"]')`	누른 곳에서 Delete 버튼을 찾습니다 (목록 하나에 위임, 34번과 같습니다)	
`button.disabled`	잠긴 버튼입니다. 브라우저는 잠긴 버튼의 클릭을 보내지 않지만 한 번 더 확인합니다	EDITOR
`const ok = await confirmDialog({…})`	사용자가 고를 때까지 기다립니다	true / false
`api.delete('/projects/' + id)`	DELETE 요청입니다. 성공하면 204(본문 없음, `null`)입니다	
`data.items.length === 0 && page > 1`	지운 뒤 그 쪽이 비었고, 1쪽이 아닌 경우입니다	2쪽의 마지막 한 줄

4. 흐름

1. Delete 버튼이 아니거나 잠겨 있으면 끝냅니다. 맞으면 그 줄의 번호와 제목을 읽습니다.
2. 확인창을 띄우고, 취소하면 끝냅니다.
3. 버튼을 잠그고 `DELETE`를 보냅니다.
4. 실패하면 버튼을 풀고 알림창을 띄우며, 404이면 목록을 다시 받습니다.
5. 성공하면 같은 조건으로 목록을 받습니다. 그 쪽이 비었고 1쪽이 아니면 앞 쪽으로 가고, 아니면 그대로 그립니다.

5. 왜 이렇게

선택	이유
HTML 모달 대신 `confirmDialog`	2026-10-07 결정(FIX-01)입니다. 모든 확인창을 같은 모양으로 맞추고, 휴대폰에서는 아래쪽 시트로 보여 줍니다
번호와 제목을 **누른 줄**에서 바로 읽음	34번의 `selected`는 다른 이벤트 처리가 먼저 불려야 채워집니다. 이벤트 처리 순서에 기대지 않으려는 것입니다

선택	이유
처음 커서는 Cancel에 둠 (`danger`)	되돌리기 어려운 일이므로, Enter 한 번에 지워지지 않게 합니다
실패는 알림창으로 보여 줌	모달이 없어졌으므로 문구 칸(`#delete-project-error`)도 없습니다
빈 쪽을 **그리지 않고** 앞 쪽으로 감	처음에는 `refreshList`로 빈 2쪽을 그린 뒤 1쪽으로 갔는데, 그사이 "No projects yet"이 잠깐 보였습니다. 받은 결과를 먼저 보고 결정하도록 고쳤습니다 (시험 03에서 빈 안내가 한 번도 보이지 않았음을 확인했습니다)
404이면 목록을 다시 받음	지워진 줄이 남지 않게 합니다 (40번과 같습니다)
누른 뒤에는 버튼을 잠금	확인한 뒤 응답이 오기 전에 두 번 눌리는 것을 막습니다 (템플릿)

6. 시험 (실제 myProject.html, 실제 서버, 7개 + 34번 다시)

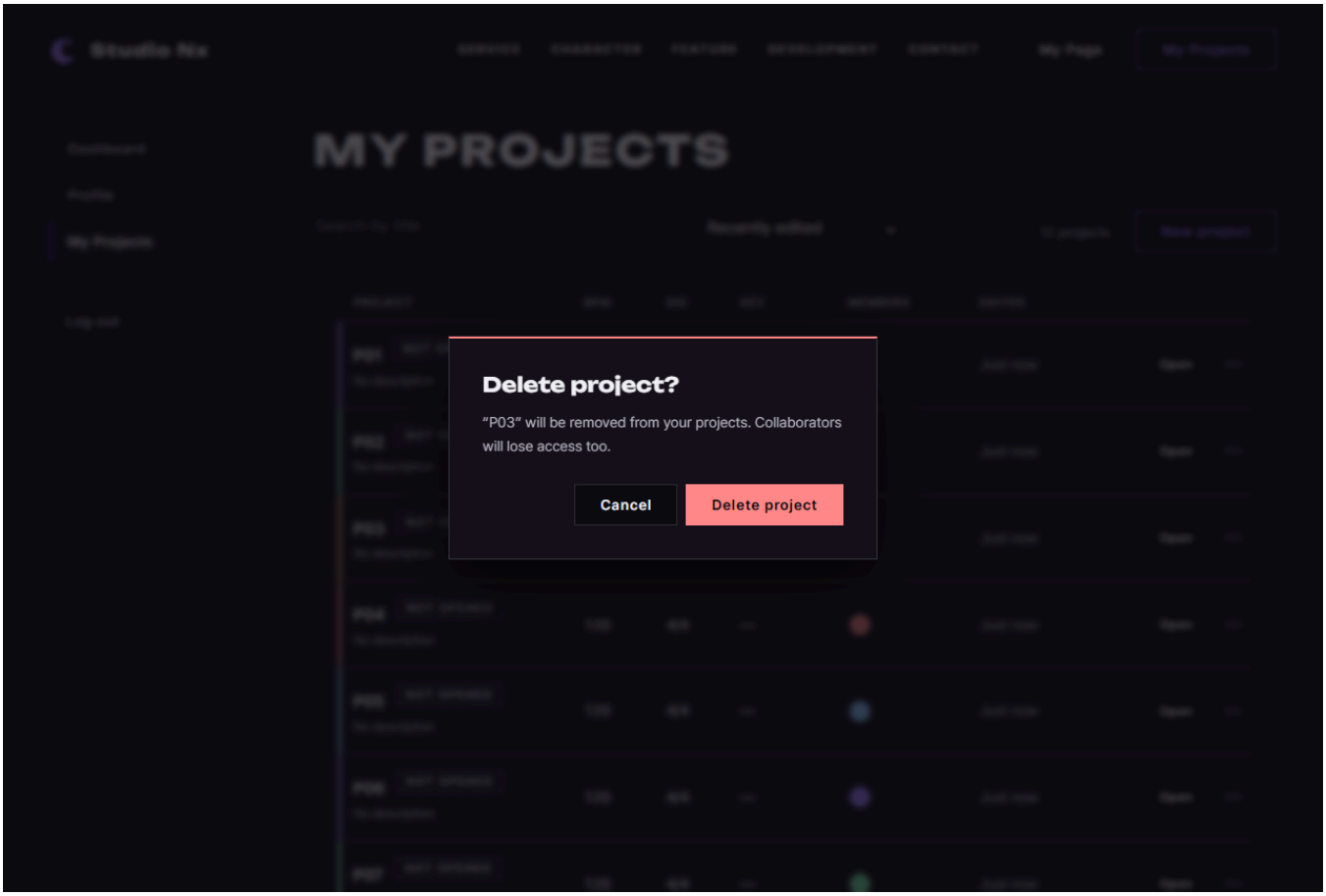
프로젝트 12개(`P01`~`P12`)를 이름순으로 그리고 …` → Delete를 눌렀습니다.

#	한 일	결과
01	P03에서 Delete	확인창의 제목, 문장, 버튼 글자가 맞습니다
01b	Cancel	DELETE는 0번이고, 12개가 그대로이며, 창이 닫힙니다
02	다시 Delete → Delete project	DELETE가 1번 가고, `11 projects`가 되며, P03이 사라지고 P11이 올라옵니다. 서버에서 검색해도 0개입니다
03	2쪽(P12 한 줄)에서 P12 삭제	1쪽 10개가 나오고, 주소는 `?sort=title`이며, 페이지 줄은 숨겨집니다. **빈 안내는 한 번도 보이지 않았습니다**
04	EDITOR 줄(가짜)의 Delete	확인창이 뜨지 않고, DELETE는 0번입니다
05	서버 500 (가짜)	알림창 `Couldn’t delete`와 `Something went wrong. Please try again.`이 나오고, 버튼이 다시 켜지며, 목록은 그대로입니다
06	확인창을 띄운 채 다른 곳에서 P02를 지운 뒤 Delete project	알림창 `Project not found.`가 나오고, 목록을 다시 받아서 P02가 사라집니다

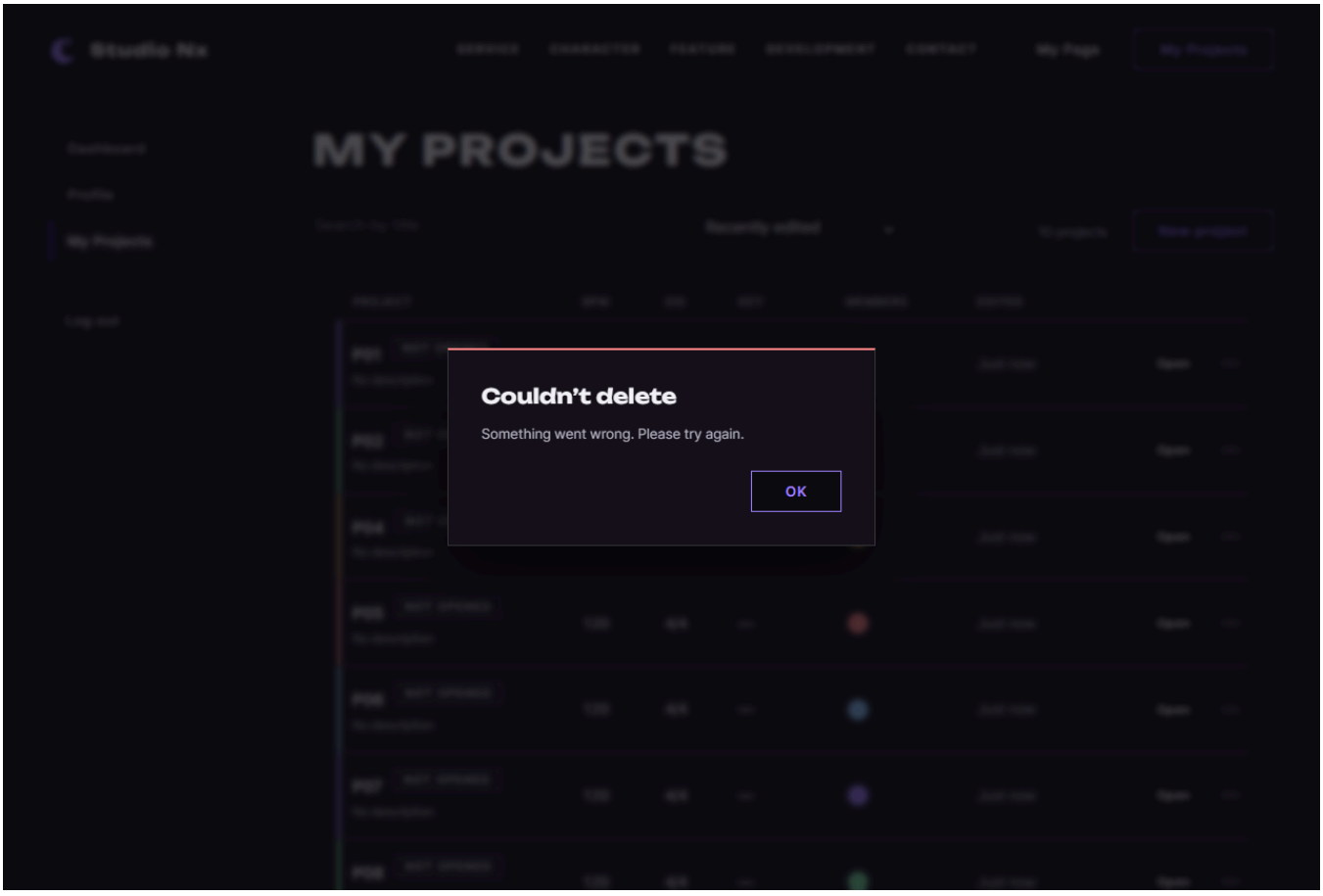
34번 다시: 삭제 모달이 없어졌으므로 34번의 시험 03을 "Delete를 누르면 확인창이 뜨고 `selected`가 그 프로젝트"로 바꿔서 7개를 다시 돌렸습니다. 모두 통과했습니다 (제목 `Neon "Rain" <x>`도 확인창에 글자 그대로 나왔습니다).

캡처

확인창:


MY PROJECTS
Delete project?
“P03” will be removed from your projects. Collaborators will lose access too.
Cancel
Delete project

서버 오류:


MY PROJECTS
Couldn’t delete
Something went wrong. Please try again.
OK

JS 42 — `loadInitialList()` · 페이지 시작

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면: 페이지를 열 때 (로그인 확인, 첫 목록, 로그아웃 버튼) (PRD 01 FR-123)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	41 `initDeleteProject()` · (P1-14 끝, 개요 51)

1. 하는 일

myProject.html을 **열었을 때** 할 일을 마무리합니다. 이 함수가 생기기 전에는 HTML의 예시 줄 7개가 그대로 보였고, 로그인하지 않아도 페이지가 열렸습니다.

주소로 열기	결과
로그인하지 않은 채로 `myProject.html`	`home.html#login?next=%2FmyProject.html`로 가고, 로그인하면 돌아옵니다
`myProject.html`	최근 수정순으로 10개, `12 projects`, `Prev 1* 2 Next`가 나오고, 예시 줄은 없습니다
`?query=p1&sort=title`	검색 칸은 `p1`, 정렬은 Name이고, `P10, P11, P12`가 나옵니다
`?page=9` (2쪽까지 있음)	**마지막 쪽(2)**으로 가고, 주소는 `?page=2`가 됩니다
`?sort=random&page=abc`	정렬은 Recently edited, 1쪽이 되고, 주소는 깨끗해집니다 (30번이 고칩니다)
Log out 버튼	로그아웃한 뒤 `home.html`로 갑니다 (P1-10 27 `initLogoutButton`)

함께 바꾼 것

- 파일 맨 아래의 시작 코드에 있던 두 TODO를 채웠습니다: `if (await requireLogin()) await loadInitialList();`
- 비어 있던 `initLogoutButton` 템플릿 함수를 지우고, `core/auth-ui.js`의 함수를 import했습니다 (P1-10에서 세 화면 공용으로 만든 것). 쓰지 않게 된 `logout` import도 뺐습니다.

2. 언제 쓰나

호출하는 곳	화면
`myproject.js` 맨 아래 (페이지를 열 때 한 번)	로그인을 확인한 뒤 첫 목록을 받습니다

시작 순서 (파일 맨 아래)

```
await includePartials();          // 헤더 · 푸터 · 안내 모달 끼우기 (아래 init 들이 그 요소를 찾음)

initAuthState();
initModalFallback();
initFormAlerts();
initSearch();                     // 33
initNewProjectForm();             // 39
initRowMenus();                   // 34
initDesktopOnlyLinks(document.getElementById('project-list'));   // 38
initEditProjectForm();            // 40
```

```
initDeleteProject();              // 41
initLogoutButton();               // core/auth-ui.js (P1-10 27) — 세 화면 공용

// 로그인 확인 → 첫 목록 (로그인 안 했으면 home.html#login?next=… 로 옮겨 감)
if (await requireLogin()) await loadInitialList();
```

버튼과 폼의 이벤트 처리를 먼저 걸고, 서버를 기다리는 일(로그인 확인, 목록)은 **맨 끝**에 둡니다. 그래서 기다리는 동안에도 버튼이 반응합니다.

3. 코드

import (`myproject.js` 맨 위)

```
import { initAuthState, initLogoutButton } from '../core/auth-ui.js';
import { requireLogin, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio, openBlankStudio } from '../co
re/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts, confirmDialog, showAlert } from '../core/dialog.js';
```

`initLogoutButton`을 추가하고 `logout`을 뺐습니다. 이 함수는 같은 파일의 `refreshList`(33)와 `current`(30)를 쓰고, 시작 코드는 `requireLogin`(P1-02)과 `initLogoutButton`(P1-10)을 씁니다.

함수

```
export async function loadInitialList() {
  const list = document.getElementById('project-list');
  const count = document.getElementById('project-count');
  const pager = document.getElementById('project-pager');
  if (list) list.textContent = '';
  if (count) count.textContent = '';
  if (pager) pager.hidden = true;

  const params = new URLSearchParams(location.search);
  const options = {
    query: params.get('query') || '',
    sort: params.get('sort') || 'updated',
    page: params.get('page') || 1,
  };
  let data = await refreshList(options);
  if (data && data.items.length === 0 && data.total > 0) {
    options.page = Math.ceil(data.total / data.size);   // 없는 쪽 → 마지막 쪽
    data = await refreshList(options);
  }
  if (!data) return null;

  const query = document.getElementById('project-query');
  const sort = document.getElementById('project-sort');
  if (query) query.value = current.query;
  if (sort) sort.value = current.sort;
```

```
  return data;
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
const params = new URLSearchParams(location.search);
const options = {
  query: params.get('query') ?? '',
  sort: params.get('sort') ?? 'updated',
  page: params.get('page') ?? 1,
};
document.getElementById('project-query')?.setAttribute('value', current.query);
```

최신 문법	실제 코드	뜻
`a ?? b`	`a || b`	없으면(null) 기본값을 씁니다. 여기서는 빈 문자열도 기본값으로 바꿔야 하므로 `||`가 오히려 맞습니다
`el?.…`	`if (el) …`	있을 때만 실행합니다

문법 풀이

import 문법	뜻
`import { initAuthState, initLogoutButton } from '../core/auth-ui.js'`	로그아웃 버튼 함수입니다 (P1-10 27). userInfo, myPage와 같은 함수를 씁니다
`import { requireLogin } from '../core/auth.js'`	로그인 상태면 `true`를, 아니면 로그인 화면으로 보내고 `false`를 돌려줍니다 (P1-02)

식	뜻	예
`new URLSearchParams(location.search)`	주소의 `?` 뒷부분을 칸별로 나눕니다	`?query=p1&sort=title`
`params.get('query') || ''`	없으면 빈 문자열을 씁니다	`null` → `''`
`Math.ceil(data.total / data.size)`	마지막 쪽 번호입니다	12 / 10 → 2
`if (await requireLogin()) await loadInitialList();`	로그인을 확인한 **뒤에만** 목록을 요청합니다	
맨 위의 `await` (모듈 맨 바깥)	`type="module"` 스크립트에서는 함수 밖에서도 `await`를 쓸 수 있습니다	

4. 흐름

1. 예시 줄과 개수를 지우고 페이지 줄을 숨깁니다 (받는 동안 가짜 목록이 보이지 않게 합니다).
2. 주소에서 검색어, 정렬, 페이지를 읽어서 목록을 받습니다 (`refreshList`; 틀린 값은 30번이 고칩니다).
3. 받은 쪽이 비었는데 전체 개수는 있으면 마지막 쪽을 다시 받습니다.
4. 검색 칸과 정렬 칸을 실제로 쓴 값(`current`)으로 맞춥니다.

5. 왜 이렇게

선택	이유
주소에서 읽음	30번이 주소에 남긴 값입니다. 새로고침, 뒤로 가기, 복사한 주소, 페이지 링크를 새 탭으로 연 경우(32)에도 같은 목록이 나옵니다
칸은 `current`로 맞춤	주소의 틀린 값(`sort=random`)이 아니라 실제로 쓴 값을 보여 줍니다
없는 쪽이면 마지막 쪽으로	프로젝트를 지운 뒤 북마크해 둔 `?page=9` 같은 주소로 와도 빈 화면이 아니라 마지막 목록이 나옵니다
받기 전에 예시 줄을 지움	HTML 예시(Summer Loop 등)가 실제 목록처럼 잠깐 보이지 않게 합니다
로그인 확인을 먼저 함	로그인하지 않은 사람에게 목록을 요청했다가 401을 받고 로그인으로 이동하는 일이 두 번 일어나지 않게 합니다 (userInfo와 같은 모양)
`initLogoutButton`은 import함	P1-10에서 세 화면 공용으로 만들어 둔 함수입니다. 템플릿 주석대로 이 파일의 빈 함수는 지웠습니다

6. 시험 (실제 myProject.html을 주소로 열기, 실제 서버, 7개 + 앞의 시험 다시)

이번에는 함수를 직접 부르지 않고 **페이지를 주소로 열기만** 했습니다.

#	한 일	결과
01	로그인하지 않은 채로 열기	`home.html#login?next=%2FmyProject.html`로 갑니다
02	로그인하고 프로젝트 12개를 만든 뒤 열기	`P12 … P03` 10줄, `12 projects`, `Prev 1* 2 Next`가 나오고, 검색 칸은 비어 있으며, 정렬은 updated이고, 예시 줄은 없습니다
03	`?query=p1&sort=title`	검색 칸 `p1`, 정렬 title, `P10, P11, P12`, `3 projects`가 나오고, 페이지 줄은 숨겨집니다
04	`?page=9`	`P02, P01`, `Prev 1 2* Next`가 나오고, 주소는 `?page=2`가 됩니다
05	`?sort=random&page=abc`	정렬 updated, 1쪽이 되고, 주소는 `myProject.html`입니다
06	Log out 클릭	`home.html`로 갑니다
07	로그아웃한 뒤 다시 열기	로그인 화면으로 갑니다

앞의 시험 다시 (이제 페이지가 스스로 목록을 받으므로): 30번(14개), 33번(6개), 39번(7개), 41번(7개)을 다시 돌려서 결과가 모두 그대로임을 확인했습니다.

캡처: 로그인한 뒤 열었을 때

